Fisher forecasts from cached derivatives

How the derivatives are computed and stored, and why every FoM, error and shift is recomputed in seconds

November 2026

1 Summary

The question. The project's Fisher notebooks report figures of merit (FoMs), parameter errors and parameter shifts for four bias models, up to four linear-bias priors, seven scale cuts and four bin layouts, under dozens of assumed truths. One evaluation of the data vector runs CAMB, the code that computes the matter power spectrum, and CosmoLike, the code that projects it onto the sky for every thin bin; it takes seconds to tens of seconds. How, then, can a change of model, prior or scale cut cost only seconds?

The answer.

1. A Fisher forecast only needs the derivatives of the data vector at one point (Section 2).
2. The paper model is built from per-bin quantities – the biases, two counterterms and two stochastic amplitudes of every thin bin – tied to about twenty group-level parameters by calibrations, offsets and tilts (Section 3).
3. The base is one derivative column for each elementary knob: each cosmological parameter, each velocity dispersion, and each bias quantity in each thin bin. A model is a recipe that moves several knobs at once; written as a matrix M, it turns the base B into the model's derivatives, $\mathsf{D} = \mathsf{BM}$, and a new model is a new M (Section 4).
4. Each layout's base is computed once and stored on disk (Section 5).
5. A scale cut keeps some rows of the data vector. Each cut is reduced once to a small matrix G (Section 6), and every Fisher matrix, FoM and shift is then a product of small matrices (Section 7).
6. A derivative needs more numerical accuracy than a χ^2, and a convergence study fixes the settings every published number uses (Section 8).
7. New evaluations are needed only for a term outside the base, a new fiducial point, or effects beyond first order (Section 9). Section 10 lists the files and when each must be recomputed.

The setup. The data are the DESI band powers C_ℓ^{gg}: the angular power spectrum of galaxy clustering, averaged in 53 bands of multipole ℓ. They are computed without the Limber approximation, which ties each multipole to a single wavenumber and fails for thin bins, for six tracer groups g (BGS, LRG1, LRG2, LRG3, ELG1 and ELG2) on four layouts: bins of equal comoving width $\Delta d_{\rm com} = 62$ Mpc, and bins of fixed redshift width $\Delta z = 0.02$, 0.05 and

0.2. Bin i has mean redshift $\bar{z}_i$, and $D(z)$ is the linear growth factor, $D(0) = 1$. The bias model and its priors are those of the correlated-prior note, `notes/correlated_bias_prior_v2`: the paper model, a one-loop bias model with per-group offsets and tilts under the pessimistic linear-bias prior, which allows correlated wiggles of the bin biases around a straight line in redshift (Section 3).

2 Why derivatives at one point are enough

Let $m(p)$ be the expected data vector at the parameters p, and p_0 the fiducial point, the parameter values around which the forecast is made. The vector m has n entries: the band powers of every bin's auto spectrum and of every measured cross spectrum between two bins, after 795 entries that the data format reserves for cosmic-shear blocks and that stay zero for DESI alone. C is the covariance of the data: how the measured band powers scatter around m and how their errors correlate.

Near p_0 the data vector is close to a straight line in the parameters,

$$m(p) \;\simeq\; m(p_0) + \mathsf{D}\,(p - p_0), \qquad \mathsf{D}_{rj} = \left.\frac{\partial m_r}{\partial p_j}\right|_{p_0}, \tag{1}$$

with r one row, that is one band power. With Gaussian data errors and Gaussian priors, the probability of the parameters given the data is then a Gaussian in p, whose inverse covariance is the Fisher matrix

$$F \;=\; \mathsf{D}^{\mathsf{T}}\mathsf{C}^{-1}\mathsf{D} \;+\; F_{\text{prior}}. \tag{2}$$

F_{prior} is the priors' contribution: a Gaussian prior of width w adds $1/w^2$ to its parameter's diagonal entry, and a flat prior adds nothing.

The inverse F^{-1} is the forecast covariance of the parameters. $\sigma_j = [(F^{-1})_{jj}]^{1/2}$ is the marginalized error of parameter j, its error when every other parameter is free. $\mathrm{FoM}(w_0, w_a) = [\det \mathsf{C}_w]^{-1/2}$ is the inverse area of the (w_0, w_a) error ellipse, with C_w the (w_0, w_a) block of F^{-1} and $w(a) = w_0 + w_a(1 - a)$ the dark-energy equation of state (a the scale factor). $\mathrm{FoM}(\Omega_m, 10^9 A_s)$ is defined the same way, with Ω_m the matter density today in units of the critical density and A_s the amplitude of the primordial curvature power spectrum.

The model enters F only through D, at one point. The covariance is Gaussian, computed at the one-loop fiducial (the fiducial point of the one-loop bias model, Section 5.2) for 14,000 square degrees, with its inverse rescaled by $f_A = 12{,}300/14{,}000$ to the survey area: a Gaussian covariance is inversely proportional to the area (more sky, more independent modes, smaller errors), so the smaller survey multiplies the covariance by $1/f_A$ and its inverse by $f_A \approx 0.879$ (`invcov_scale` in the code and the caches). It does not depend on the model being fitted. Its own numerical accuracy was audited when it was built: its non-Limber distance grid sits where a finer grid moves the adjacent-bin cross band powers by less than 0.4%, while the next coarser grid misses them by up to 15–30% (`covariance/README.md` records the measured convergence).

3 The model being forecast

The quantities, bin by bin. Everything the notebooks fit is built from a small set of quantities:

- seven cosmological parameters, with flat priors: $10^9 A_s$, the spectral index n_s, the Hubble constant H_0, the baryon density Ω_b, Ω_m, w_0 and w_a;
- seven velocity dispersions $\sigma_{v,1}, \ldots, \sigma_{v,7}$, with Gaussian priors, one per set of bins that shares a value. They smooth each bin's redshift distribution for the random velocities of galaxies (the Fingers-of-God effect). The sets follow the six tracer groups, except that BGS takes two: on $\Delta d_{\rm com} = 62$ Mpc the dispersions cover bins 1–10 and 11–18 (BGS), 19–29 (LRG1), 30–38 (LRG2), 39–49 (LRG3), 50–61 (ELG1) and 62–76 (ELG2);
- separately in every thin bin $i = 1, \ldots, N$: the linear bias $b_{1,i}$; the quadratic and tidal biases $b_{2,i}$ and $b_{s^2,i}$, the galaxies' responses to the squared density and to the tidal field; the counterterms $\alpha_{0,i}$ and $\alpha_{2,i}$, terms proportional to k^2 that absorb small-scale physics; and the stochastic amplitudes $\mathrm{SN}_{0,i}$ and $\mathrm{SN}_{2,i}$, noise-like terms from the discreteness of galaxies.

The per-bin power spectrum. The real-space galaxy power spectrum of bin i is

$$P_{gg,i}(k) \;=\; b_{1,i}^2\, P_{\rm mm}(k) \;+\; P_{\rm loop}(k;\, b_{1,i}, b_{2,i}, b_{s^2,i}) \;+\; \alpha_{0,i}\, k^2 P_{\rm mm}(k), \tag{3}$$

with $P_{\rm mm}$ the nonlinear matter power spectrum and $P_{\rm loop}$ the one-loop terms, the next order of perturbation theory beyond linear, which carry $b_{2,i}$ and $b_{s^2,i}$. $\alpha_{0,i}\, k^2 P_{\rm mm}$ is a counterterm, in $h^{-2}\,\mathrm{Mpc}^2$: a term that absorbs small-scale physics the model does not compute. In redshift space the model adds the counterterm $\alpha_{2,i}$, in the same units, a term proportional to $k^2\mu^2$ times the linear power (μ the cosine of the angle to the line of sight), and the stochastic terms $\mathrm{SN}_{0,i}$ (a white noise beyond the Poisson one) and $\mathrm{SN}_{2,i}\, k^2\mu^2$ (its line-of-sight counterpart). Section 5.4 writes out how every term reaches the projected band powers; the full physics is in the higher-order note, `notes/higher_order_bias`, and the correlated-prior note, `notes/correlated_bias_prior_v2`.

How the fit parameterizes the bins. No fit gives every bin its own free quantities; the parameter files tie the bins to their groups:

- the linear bias follows one amplitude B_g per group, $b_{1,i} = B_g / D(\bar{z}_i)$: the group's clustering amplitude $b_1 D$ is constant across its bins;
- $b_{2,i}$ follows $b_{1,i}$ through a calibration, the relation $b_2^{\rm L}(b_{1,i})$ fitted to simulations by Lazeyras et al. [1], plus one offset $\Delta_{2,g}$ per group; $b_{s^2,i}$ follows the co-evolution relation, the prediction for galaxies that form with no tidal bias and then move with the matter, $b_{s^2,i} = -\frac{4}{7}(b_{1,i} - 1) + \Delta_{s,g}$, with its own group offset;
- $\alpha_{0,i}$, $\alpha_{2,i}$, $\mathrm{SN}_{0,i}$ and $\mathrm{SN}_{2,i}$ take one value per group: $\alpha_{0,g}$, $\alpha_{2,g}$, $\mathrm{SN}_{0,g}$ and $\mathrm{SN}_{2,g}$;
- the tilts let a group value drift linearly in redshift: $\Delta_{2,g}$ becomes $\Delta_{2,g} + t_{2,g}(\bar{z}_i - \bar{z}_g)$, with $t_{2,g}$ the tilt and $\bar{z}_g$ the group's mean bin redshift, and likewise for $\Delta_{s,g}$, $\alpha_{0,g}$ and $\alpha_{2,g}$; the stochastic amplitudes never tilt;
- the linear bias is fitted under the pessimistic prior of the correlated-prior note: one amplitude per bin, with a free amplitude and tilt per group and correlated residuals around them. (That note defines three more linear-bias priors, from growth-tied to free; the machinery below takes any of them.)

The paper model. The model this note forecasts is the paper model: the one-loop bias model above with, per group, the offsets $\Delta_{2,g}$ and $\Delta_{s,g}$, the counterterm amplitudes $\alpha_{0,g}$ and $\alpha_{2,g}$, the stochastic amplitudes $\mathrm{SN}_{0,g}$ and $\mathrm{SN}_{2,g}$ and the four tilts, under the pessimistic linear-bias prior; every group parameter carries the Gaussian prior width of the correlated-prior note. The same machinery builds the recipe matrix of any other model assembled from the same quantities, which the exploration notebooks fit and the correlated-prior and higher-order notes describe.

Why the forecast needs per-bin quantities. A chain samples only the group-level parameters above, about twenty in all. The forecast, though, must price choices that differ in exactly how the bins are tied together: one amplitude per group, per bin, or correlated in between; offsets with or without tilts. All of them are recipes on the same per-bin quantities, so the object worth precomputing is the response of the data vector to each quantity in each single bin. That is the base of the next section.

4 The base, and every model as a matrix on it

4.1 What the base is

The base is the list of elementary knobs the code turns by hand: each cosmological parameter, each velocity dispersion, and, separately in every thin bin, each of the seven bias quantities of Section 3. For each knob, the code computes how every band power changes when that knob alone moves: one derivative column, a vector of n numbers. These are the base columns, and they are what the notebooks precompute and store (Section 7.1).

The base matrix B puts the base columns side by side. A model parameter, such as a group offset, a tilt or a per-bin amplitude, is a recipe that moves several knobs at once. The chain rule turns that recipe into a column of the model's derivative matrix: how m changes with p_j is the sum, over the knobs, of how m changes with the knob times how the knob changes with p_j,

$$\frac{\partial m}{\partial p_j} \;=\; \sum_a \frac{\partial m}{\partial \kappa_a}\,\frac{\partial \kappa_a}{\partial p_j}, \qquad \text{that is} \qquad \mathsf{D} \;=\; \mathsf{B}\,\mathsf{M}, \qquad \mathsf{M}_{aj} = \frac{\partial \kappa_a}{\partial p_j}, \tag{4}$$

with κ_a the value of knob a and p_j the model's parameter j. M is the recipes written as numbers: M_{aj} is how much knob a moves per unit of parameter j. Section 4.2 works one out.

There is no single b_1. Every bias knob belongs to one bin. "The b_1 columns" are therefore N columns, $\partial m/\partial b_{1,1}, \ldots, \partial m/\partial b_{1,N}$, and the same holds for each of the seven bias families $\{b_{1,i}\}$, $\{b_{2,i}\}$, $\{b_{s^2,i}\}$, $\{\alpha_{0,i}\}$, $\{\alpha_{2,i}\}$, $\{\mathrm{SN}_{0,i}\}$ and $\{\mathrm{SN}_{2,i}\}$, $i = 1, \ldots, N$. In order, B has

$$n_{\mathrm{base}} \;=\; \underbrace{7}_{\text{cosmology}} + \underbrace{7}_{\sigma_v} + \underbrace{7N}_{7\ \text{families}\times N\ \text{bins}} + \underbrace{1}_{\text{check}} \;=\; 7N + 15 \tag{5}$$

columns; the check column is explained in Section 9. On $\Delta d_{\mathrm{com}} = 62$ Mpc there are $N = 76$ bins, so B has $7 + 7 + 532 + 1 = 547$ columns of $n = 10{,}494$ numbers each. Table 1 gives every layout.

layout	bins N	cross spectra	entries n	n_{base}	kept, shipped mask
$\Delta d_{\text{com}} = 62$ Mpc	76	107	10,494	547	5,674
$\Delta z = 0.02$	90	133	12,614	645	7,621
$\Delta z = 0.05$	36	52	5,459	267	2,982
$\Delta z = 0.2$	10	13	2,014	85	744

Table 1: The sizes of one layout's Fisher problem. Cross spectra: the pairs of bins whose cross spectrum is in the data vector. n: entries of the data vector. $n_{\text{base}} = 7N + 15$: columns of the base. Last column: the band powers the shipped mask keeps (the mask of the data files, before any scale cut), the largest set of rows any cut uses.

4.2 A toy example

The setup. Take one group g of $N = 3$ bins, with mean redshifts $\bar{z}_1$, $\bar{z}_2$, $\bar{z}_3$, their mean $\bar{z}_g$, and $D_i \equiv D(\bar{z}_i)$. Keep one band of each auto spectrum and of the two cross spectra of neighboring bins, so the data vector has $n = 5$ entries,

$$m = \left(C^{11}, C^{22}, C^{33}, C^{12}, C^{23}\right)^{\mathsf{T}}. \tag{6}$$

Keep one cosmological parameter θ and two bias families. The knobs are θ, $b_{1,1}$, $b_{1,2}$, $b_{1,3}$, $b_{2,1}$, $b_{2,2}$ and $b_{2,3}$, so $n_{\text{base}} = 7$.

The base matrix. Name the base columns $y = \partial m/\partial\theta$, $u_i = \partial m/\partial b_{1,i}$ and $v_i = \partial m/\partial b_{2,i}$. Then

$$\mathsf{B} = \begin{array}{c} \\ C^{11} \\ C^{22} \\ C^{33} \\ C^{12} \\ C^{23} \end{array} \begin{array}{c} \begin{array}{ccccccc} y & u_1 & u_2 & u_3 & v_1 & v_2 & v_3 \end{array} \\ \begin{pmatrix} * & * & 0 & 0 & * & 0 & 0 \\ * & 0 & * & 0 & 0 & * & 0 \\ * & 0 & 0 & * & 0 & 0 & * \\ * & * & * & 0 & * & * & 0 \\ * & 0 & * & * & 0 & * & * \end{pmatrix} \end{array}, \tag{7}$$

where $*$ marks a nonzero entry. The column u_1 vanishes on C^{22}, C^{33} and C^{23}, because the bias of bin 1 does not enter those spectra. A cross spectrum, such as C^{12}, depends on the biases of both its bins.

The model. The model has the parameters $p = (\theta, \Delta_{2,g}, t_{2,g}, \beta_1, \beta_2, \beta_3)$, so $n_{\text{par}} = 6$, and sets the knobs by two recipes:

$$b_{1,i} = \frac{\beta_i}{D_i}, \qquad b_{2,i} = b_2^{\text{L}}(b_{1,i}) + \Delta_{2,g} + t_{2,g}\,\zeta_i, \qquad \zeta_i \equiv \bar{z}_i - \bar{z}_g. \tag{8}$$

Here β_i is the amplitude of bin i, and b_2^{L} the calibration of Section 3, which predicts $b_{2,i}$ from $b_{1,i}$. $\Delta_{2,g}$ is the group's offset from that calibration and $t_{2,g}$ its tilt in redshift. Let $s_i \equiv b_2^{\text{L}\prime}(b_{1,i})$ be the slope of the calibration.

The recipes as numbers. M_{aj} is how much knob a moves per unit of parameter j:

1. θ moves only θ. (y is computed by moving θ with the amplitudes β_i held fixed, which is how the model holds them, so θ needs no chain rule.)

2. $\Delta_{2,g}$ moves $b_{2,1}$, $b_{2,2}$ and $b_{2,3}$ by 1 each.

3. $t_{2,g}$ moves $b_{2,i}$ by ζ_i.

4. β_i moves $b_{1,i}$ by $1/D_i$ and, through the calibration, $b_{2,i}$ by s_i/D_i; the other bins do not move.

Each item is one column of

$$\mathsf{M} = \begin{array}{c} \\ \theta \\ b_{1,1} \\ b_{1,2} \\ b_{1,3} \\ b_{2,1} \\ b_{2,2} \\ b_{2,3} \end{array} \begin{array}{c} \begin{array}{cccccc} \theta & \Delta_{2,g} & t_{2,g} & \beta_1 & \beta_2 & \beta_3 \end{array} \\ \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1/D_1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1/D_2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1/D_3 \\ 0 & 1 & \zeta_1 & s_1/D_1 & 0 & 0 \\ 0 & 1 & \zeta_2 & 0 & s_2/D_2 & 0 \\ 0 & 1 & \zeta_3 & 0 & 0 & s_3/D_3 \end{pmatrix} \end{array}. \tag{9}$$

The multiplication. Column j of $\mathsf{D} = \mathsf{BM}$ is the sum of the columns of B, weighted by column j of M:

$$\begin{aligned} \frac{\partial m}{\partial \theta} &= y, & \frac{\partial m}{\partial \Delta_{2,g}} &= v_1 + v_2 + v_3, \\ \frac{\partial m}{\partial t_{2,g}} &= \zeta_1 v_1 + \zeta_2 v_2 + \zeta_3 v_3, & \frac{\partial m}{\partial \beta_i} &= \frac{u_i + s_i v_i}{D_i}. \end{aligned} \tag{10}$$

Written out entry by entry, the offset column is

$$\frac{\partial m}{\partial \Delta_{2,g}} = \left(\frac{\partial C^{11}}{\partial b_{2,1}}, \frac{\partial C^{22}}{\partial b_{2,2}}, \frac{\partial C^{33}}{\partial b_{2,3}}, \frac{\partial C^{12}}{\partial b_{2,1}} + \frac{\partial C^{12}}{\partial b_{2,2}}, \frac{\partial C^{23}}{\partial b_{2,2}} + \frac{\partial C^{23}}{\partial b_{2,3}} \right)^{\mathsf{T}}: \tag{11}$$

each auto spectrum responds to its own bin's $b_{2,i}$, and each cross spectrum to both of its bins'.

Another model, the same base. The free-per-bin model replaces the group offset and tilt by one offset per bin, $b_{2,i} = b_2^{\mathrm{L}}(b_{1,i}) + \Delta_{2,i}$. Each offset moves one knob, so its block on the $b_{2,i}$ rows is the 3×3 identity:

$$\mathsf{M}_{\mathrm{free}} = \begin{array}{c} \\ \theta \\ b_{1,1} \\ b_{1,2} \\ b_{1,3} \\ b_{2,1} \\ b_{2,2} \\ b_{2,3} \end{array} \begin{array}{c} \begin{array}{ccccccc} \theta & \Delta_{2,1} & \Delta_{2,2} & \Delta_{2,3} & \beta_1 & \beta_2 & \beta_3 \end{array} \\ \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1/D_1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1/D_2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1/D_3 \\ 0 & 1 & 0 & 0 & s_1/D_1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & s_2/D_2 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & s_3/D_3 \end{pmatrix} \end{array}, \qquad \frac{\partial m}{\partial \Delta_{2,i}} = v_i. \tag{12}$$

The priors.

- Group model: $1/w_g^2$ on $\Delta_{2,g}$ (a Gaussian prior of width w_g), nothing on $t_{2,g}$ (a flat prior).
- Free per bin: $1/w_g^2$ on each $\Delta_{2,i}$.
- Free per bin, $\sqrt{N}$-scaled: the same M_{free}, with $1/(3\,w_g^2)$ on each $\Delta_{2,i}$. The mean of the three offsets then has the width $w_g\sqrt{3}/\sqrt{3} = w_g$ of the group offset.

Three models, one B: the group and free models differ in M, the free model and its $\sqrt{N}$-scaled version only in F_{prior}. The base, the expensive part, never changes.

4.3 Every notebook model as a matrix on the base

The toy's recipes carry over to the real models, with seven bias families instead of two and six groups (BGS, LRG1, LRG2, LRG3, ELG1, ELG2) instead of one. Every model is a matrix M of Eq. (4) on the same B, written down without any evaluation (`variant_columns` in the model B notebook). Figure 1 draws its structure. Its columns:

- **Cosmology and σ_v**: unit vectors, as θ in the toy.
- **A group offset**: ones on the rows of the group's bins in its family's block, as $\Delta_{2,g}$ in the toy. The offsets $\Delta_{2,g}$ and $\Delta_{s,g}$ sit on the $b_{2,i}$ and $b_{s^2,i}$ blocks, and the group's $\alpha_{0,g}$, $\alpha_{2,g}$, $\mathrm{SN}_{0,g}$ and $\mathrm{SN}_{2,g}$ on the $\alpha_{0,i}$, $\alpha_{2,i}$, $\mathrm{SN}_{0,i}$ and $\mathrm{SN}_{2,i}$ blocks.
- **A tilt**: the weights $\bar{z}_i - \bar{z}_g$ on the same rows, as $t_{2,g}$ in the toy: the four tilt families of Section 3, in every group of more than one bin, with flat priors.
- **The per-bin amplitude**: as β_i in the toy, plus the tidal bias, which follows $b_{1,i}$ by the co-evolution relation of Section 3, $b_{s^2,i} = -\frac{4}{7}(b_{1,i} - 1) + \Delta_{s,g}$. The column has three entries:
$$\frac{\partial m}{\partial \beta_i} \;=\; \frac{1}{D(\bar{z}_i)}\left[\frac{\partial m}{\partial b_{1,i}} + b_2^{\mathrm{L}\,\prime}(b_{1,i})\,\frac{\partial m}{\partial b_{2,i}} - \frac{4}{7}\,\frac{\partial m}{\partial b_{s^2,i}}\right]. \tag{13}$$
- **The growth-tied prior**: one amplitude per group, $b_{1,i} = B_g/D(\bar{z}_i)$, whose column is the sum of the group's β_i columns.
- **Free per bin, and $\sqrt{N}$-scaled**: as in the toy, for all six families, on the free linear-bias prior; the widths are the group parameter's w_g, or $w_g\sqrt{N_g}$ with N_g the number of bins in the group.

On $\Delta d_{\text{com}} = 62$ Mpc the paper model's M has $n_{\text{par}} = 14 + 6\times 6 + 4\times 6 + 76 = 150$ columns: the cosmology and σ_v, six per-group families, the four tilt families, and one amplitude per bin. Any other model built from the same knobs is another M on the same B; switching between them needs no evaluation of the data vector.

5 Computing the base

The base columns are computed once per layout and cached on disk. This section is the machinery: the finite-difference stencil, the batched per-bin columns, the wrapped setter through which the steps reach CosmoLike, the exact templates of the linear terms, and what it all costs.

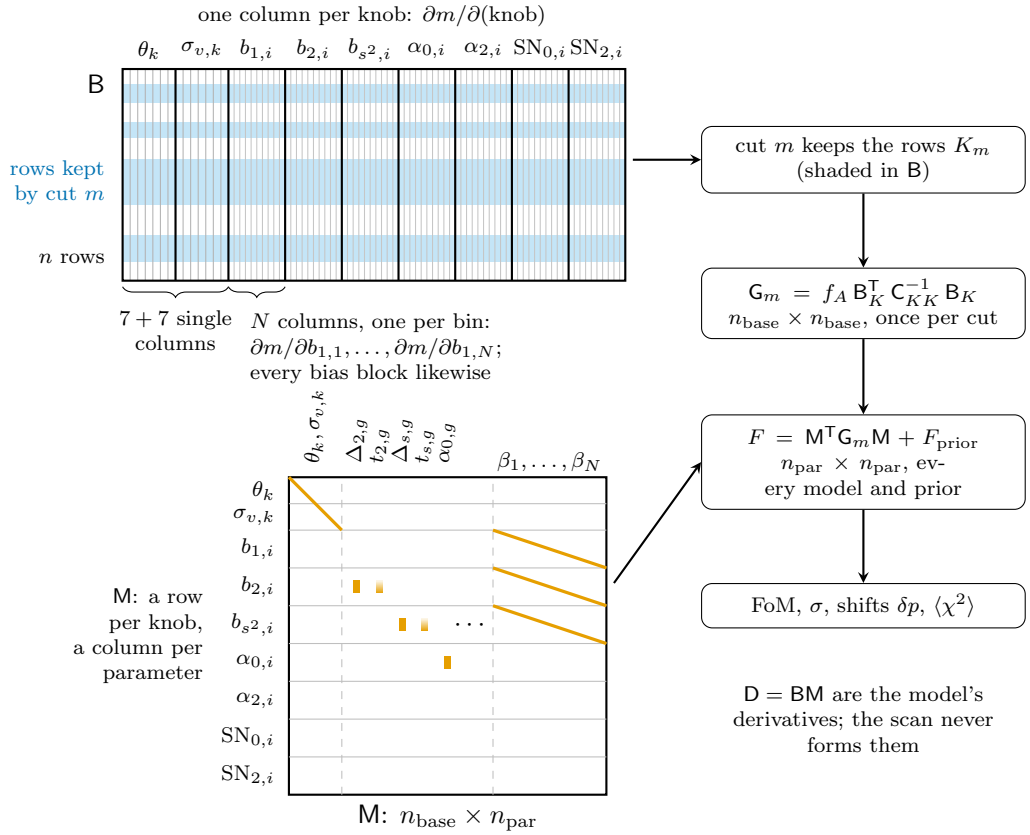


Figure 1: How the notebooks assemble a Fisher matrix. Top left: the base B of one layout. Each cosmological parameter θ_k and each velocity dispersion $\sigma_{v,k}$ has one column; every bias family is a block of N columns, one per thin bin (thin gray lines). Shaded: the rows a cut m keeps. Bottom left: a model's recipe matrix M, with a row per knob, in the column order of B, and a column per model parameter. Orange marks its nonzero entries: the identity on the cosmology and σ_v; for one group g, the offset $\Delta_{2,g}$ (ones on the group's bins in the $b_{2,i}$ block), its tilt $t_{2,g}$ (the weights $\bar{z}_i - \bar{z}_g$, drawn as a gradient), and likewise $\Delta_{s,g}$, $t_{s,g}$ and $\alpha_{0,g}$; and three diagonals for the per-bin amplitudes β_i, Eq. (13). Right: a cut is reduced once to G_m, and every model and prior at that cut is $\mathsf{M}^{\mathsf{T}}\mathsf{G}_m\mathsf{M}$ plus priors; $f_A = 12{,}300/14{,}000$ rescales the inverse covariance to the survey area (Section 2). The check column of B is not drawn.

5.1 How a derivative column is computed

A finite difference. The code cannot differentiate CAMB and CosmoLike by hand. It moves one knob by a small step ϵ, recomputes the data vector, and divides the change by the step. The simplest such finite difference, the central difference $[m(\theta_0 + \epsilon) - m(\theta_0 - \epsilon)]/(2\epsilon)$, is off by a term of order ϵ^2 times the third derivative of m: the curvature it ignores.

The five-point stencil. A stencil is the set of points at which m is evaluated, with the weights that combine them. The code uses five points, $\theta_0 \pm \epsilon$, $\theta_0 \pm 2\epsilon$ and θ_0, with the weights

$$\frac{\partial m}{\partial \theta} \simeq \frac{m(\theta_0 - 2\epsilon) - 8\, m(\theta_0 - \epsilon) + 8\, m(\theta_0 + \epsilon) - m(\theta_0 + 2\epsilon)}{12\, \epsilon}. \tag{14}$$

These weights cancel the error terms through fourth order, so the remaining error falls as ϵ^4, and the formula is exact for any dependence up to quartic. The center point has weight zero and is never evaluated, so one column costs four evaluations.

The cosmological and σ_v columns. These are stencils of the sampled parameters of the likelihood, taken by `model.derivatives` of the Fisher module (`interface/cosmolike_lssty1xdesi_fisher.py`). The step is relative, $\epsilon = h\,\theta_0$ with $h = 0.02$; a parameter whose fiducial value is zero (w_a) takes an absolute step, scaled by its prior width when its prior is Gaussian. Each evaluation is a full call of the likelihood, CAMB included, with the cache of previous results of cobaya (the framework that runs the likelihood) disabled, so that cobaya never hands back a vector saved at an earlier call in place of a fresh one (Section 5.3).

The same call also returns the stencils of the six group amplitudes B_g, which set the linear bias of every bin of group g at the fiducial point, $b_{1,i} = B_g/D(\bar{z}_i)$. They enter no Fisher matrix of the notebooks, because the per-bin columns replace them; they are kept for the chain-rule check of Section 9.

5.2 The per-bin columns, in batches

None of the four families $\{b_{1,i}\}$, $\{b_{2,i}\}$, $\{b_{s^2,i}\}$ and $\{\alpha_{0,i}\}$ of the galaxy power spectrum, Eq. (3), enters m linearly (Section 5.4), so each needs a stencil.

The stencil of one bin. For the linear bias of bin i, with every other knob at its fiducial value:

1. evaluate m four times, with $b_{1,i}$ replaced by $b_{1,i}(1 \pm \epsilon)$ and $b_{1,i}(1 \pm 2\epsilon)$;
2. combine the four vectors with the weights of Eq. (14), the step being $\epsilon\, b_{1,i}$:

$$\frac{\partial m}{\partial b_{1,i}} \simeq \frac{m\big(b_{1,i}(1-2\epsilon)\big) - 8\,m\big(b_{1,i}(1-\epsilon)\big) + 8\,m\big(b_{1,i}(1+\epsilon)\big) - m\big(b_{1,i}(1+2\epsilon)\big)}{12\,\epsilon\, b_{1,i}}. \quad (15)$$

The step of $b_{1,i}$ is relative, a fraction $\epsilon = 0.01$ of $b_{1,i}$. The steps of the other three are absolute: the four points are $b_{2,i} \pm \epsilon$ and $b_{2,i} \pm 2\epsilon$ with $\epsilon = 0.02$, the same for $b_{s^2,i}$, and $\epsilon = 5\,h^{-2}\,\mathrm{Mpc}^2$ for $\alpha_{0,i}$.

The problem: one bin at a time is wasteful. Done bin by bin, a family costs 4 evaluations per bin: $4 \times 76 = 304$ on $\Delta d_{\mathrm{com}} = 62$ Mpc, and 1,216 for the four families. Most of that work is wasted. The data vector of that layout holds 183 spectra, 76 autos and 107 crosses, and bin i enters at most six of them: its own auto spectrum and up to five cross spectra. Every other row of $\partial m/\partial b_{1,i}$ is exactly zero, and an evaluation that moves one bin leaves 177 or more spectra unchanged. Counted per spectrum, the block of one family, 183 spectra by 76 columns, is 98% zeros; in the cached arrays no column has a nonzero entry outside its own bin's spectra.

The idea: move several bins in one evaluation. Take five bins in a row whose only measured cross spectra join neighbors: 1–2, 2–3, 3–4 and 4–5. Put the bins in two batches, sets of bins moved together in one stencil:

spectrum	$\partial/\partial b_{1,1}$	$\partial/\partial b_{1,2}$	$\partial/\partial b_{1,3}$	$\partial/\partial b_{1,4}$	$\partial/\partial b_{1,5}$
C^{11}	A	0	0	0	0
C^{22}	0	B	0	0	0
C^{33}	0	0	A	0	0
C^{44}	0	0	0	B	0
C^{55}	0	0	0	0	A
C^{12}	A	B	0	0	0
C^{23}	0	B	A	0	0
C^{34}	0	0	A	B	0
C^{45}	0	0	0	B	A

Table 2: Five bins in a row, cross spectra between neighbors only. Each entry names the batch whose evaluations fill it: batch A moves bins 1, 3 and 5, batch B moves bins 2 and 4. A 0 is exactly zero, because the spectrum does not contain that bin.

- batch A moves $b_{1,i}$ in bins 1, 3 and 5 together;
- batch B moves $b_{1,i}$ in bins 2 and 4 together.

Each batch is one five-point stencil, four evaluations. Figure 2 shows batches on a real layout: each bin is a dot, a line joins two bins whose cross spectrum is in the data vector, and the number in each dot is the bin's batch. Table 2 shows which batch fills each entry of the five columns $\partial m/\partial b_{1,1}, \ldots, \partial m/\partial b_{1,5}$.

Reading the table:

- An auto spectrum C^{ii} is filled in the batch that moves bin i.
- A cross spectrum C^{ij} appears in two columns, filled from two different batches: $\partial C^{ij}/\partial b_{1,i}$ from the batch where bin i moves and bin j stays put, and $\partial C^{ij}/\partial b_{1,j}$ from the batch where j moves and i stays put.
- A 0 is not computed: a spectrum that does not contain bin i cannot depend on $b_{1,i}$, so the code sets those rows to zero.

Bin by bin, the five columns would cost $5 \times 4 = 20$ evaluations; the two batches cost $2 \times 4 = 8$.

A batch is a set of bins, not of spectra. No batch holds "the autos" or "the crosses". In the evaluations of the batch that contains bin i, bin i's auto spectrum and all of its cross spectra receive their derivative with respect to $b_{1,i}$. A cross spectrum C^{ij} is visited twice, once in the batch of each of its two bins. Figure 3c shows this on a real block.

When may two bins move together? Why not move every bin at once and keep, for each column, only its own rows?

- That works for the auto spectra: C^{ii} depends only on bin i's biases.
- It fails for the cross spectra. If bins i and j move together by δ, C^{ij} changes by $(\partial C^{ij}/\partial b_{1,i} + \partial C^{ij}/\partial b_{1,j})\,\delta$, and one evaluation cannot split the two terms.

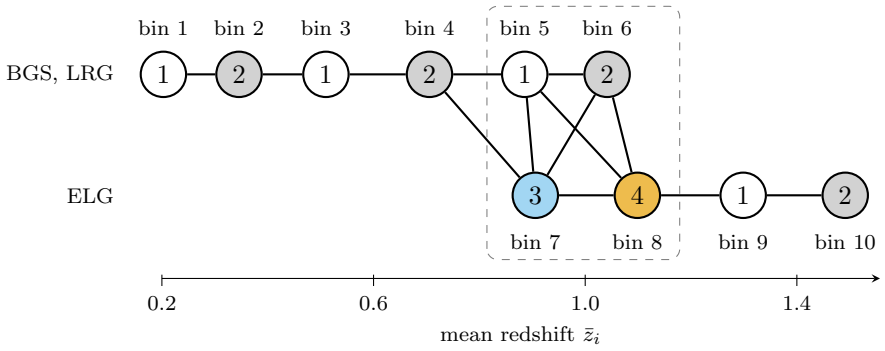


Figure 2: The cross-pair graph of the $\Delta z = 0.2$ layout, all ten bins. Each dot is a bin at its mean redshift (top row BGS and LRG, bottom row ELG), and a line joins two bins whose cross spectrum is in the data vector. The number in each dot is its batch from the greedy rule, repeated by the dot's shade. Bins 5 to 8 (dashed box: two LRG3 and two ELG1 bins, which overlap in redshift) are all joined to each other, so they need four different batches.

So two bins may share a batch only if no measured spectrum contains both. Batching is "move as many bins at once as possible, then keep each column's own rows", with the fewest batches this rule allows.

What a batch is, as a picture. Draw each bin as a dot, and join two dots with a line when the data vector contains their cross spectrum: this is the cross-pair graph (Figure 2).

- A batch is a set of dots with no line between any two of them, so its bins can move together.
- Batching the bins means giving every dot a batch number, so that two joined dots never share a number.
- The code (`pair_coloring`) uses a greedy rule: it goes through the bins in data-vector order and gives each the lowest batch number that none of its already numbered neighbors has.

Mathematicians call such batches colors, from the picture of labelling the dots so that joined dots never share a label; hence the name of the code's function.

The real layout, step by step. Take $\Delta d_{\rm com} = 62$ Mpc. Its $N = 76$ bins are numbered 1 to 76 in data-vector order, by group and, within a group, by redshift (numbers start at 1, as everywhere in this note and in Figure 3; the code counts from 0). DESI's galaxies come in six groups, ordered by redshift (Table 3). Figure 3 draws the whole layout and its batches; the steps below walk through it.

Which cross spectra are measured. The rule is simple: the data vector holds the cross spectrum of two bins when their redshift distributions $n(z)$ overlap or touch, that is, when the bins are neighbors in redshift. Bins far apart in redshift share no spectrum. For the fixed-Δz layouts this is the rule of `cross_pairs` in `scripts/make_desi_dz_case.py`. On $\Delta d_{\rm com} =$

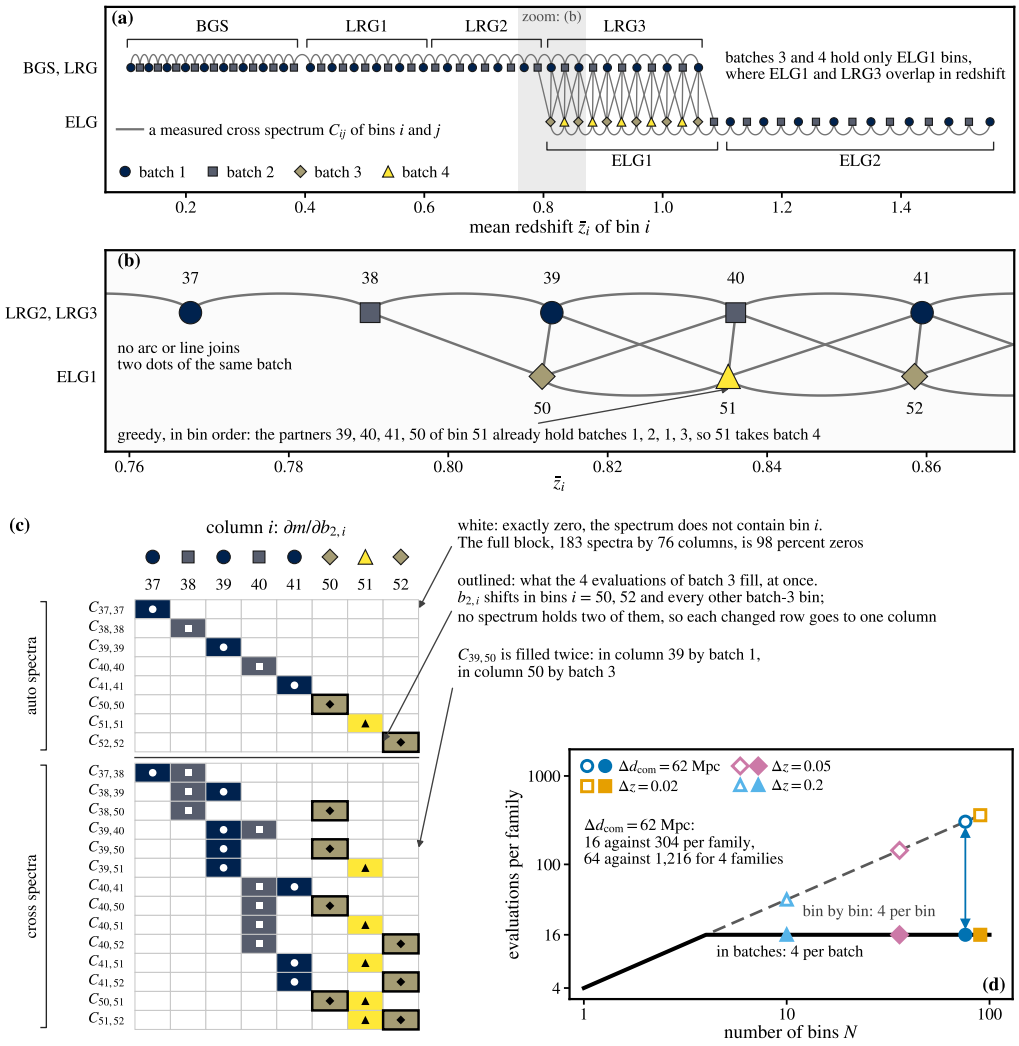


Figure 3: How the per-bin bias columns of $\Delta d_{\rm com} = 62$ Mpc are computed in batches. (a) Every bin at its mean redshift, BGS and LRG on the top row, ELG on the bottom; an arc or line is a measured cross spectrum, and the marker gives the batch. Look first at the long chains: each bin is crossed only with its neighbors, so the markers alternate between batch 1 (circle) and batch 2 (square). Then look at the overlap, where LRG3 and ELG1 cover the same redshifts: the lines between the rows are their cross spectra, and only there do batches 3 (diamond) and 4 (triangle) appear. The gray band marks the region panel (b) zooms into. (b) A zoom on that region. No arc or line joins two markers of the same batch. Follow the greedy rule at bin 51: its earlier partners 39, 40, 41 and 50 already hold batches 1, 2, 1 and 3, so it takes batch 4. (c) The block $\partial m/\partial b_{2,i}$ of those eight bins, with a row per spectrum and a column per bin. A column is nonzero only on the spectra that contain its bin; white cells are exactly zero. The outlined cells are what the four evaluations of batch 3 fill at once. A cross spectrum such as $C_{39,50}$ is filled twice: in column 39 by batch 1, and in column 50 by batch 3. (d) Evaluations per bias family against the number of bins N: bin by bin the cost grows as $4N$ (open markers, dashed line); in batches it stops at $4 \times 4 = 16$ (filled markers), whatever N.

group	galaxies	bins	mean redshifts $\bar{z}_i$
BGS	bright galaxies	1–18	0.11–0.38
LRG1	luminous red galaxies	19–29	0.41–0.60
LRG2	luminous red galaxies	30–38	0.62–0.79
LRG3	luminous red galaxies	39–49	0.81–1.06
ELG1	emission-line galaxies	50–61	0.81–1.09
ELG2	emission-line galaxies	62–76	1.11–1.55

Table 3: The six groups of $\Delta d_{\rm com} = 62$ Mpc. LRG3 and ELG1 cover the same redshifts: they overlap.

62 Mpc the pair list is built by `scripts/make_desi_pairs.py`: it starts from the 97 pairs the DESI forecast measures and adds every pair whose kernels, blurred by the Fingers-of-God smoothing, still overlap, $O_{ij} \geq 10^{-6}$ with O_{ij} the normalized overlap of the two blurred $n(z)$. That adds ten LRG3–ELG1 pairs, for 107 in all. The threshold is not a tuning: any value between 10^{-6} and 0.0277 gives the same lists, and the blur adds no pair on the fixed-Δz layouts. The cross-correlation note, `notes/cross_correlations`, derives the rule and the blur.

On $\Delta d_{\rm com} = 62$ Mpc the 107 cross spectra come in two kinds:

1. 74 join a bin to the next one along the chain of bins: neighboring thin shells, which touch at their shared edge. These include the four links at group boundaries: BGS–LRG1, LRG1–LRG2, LRG2–LRG3 and ELG1–ELG2.

2. 33 join an LRG bin to an ELG1 bin at nearly the same redshift: 32 LRG3–ELG1 pairs and one LRG2–ELG1 pair at $z \approx 0.8$. This is the overlap, explained below.

Each bin has 1 to 5 cross partners.

Along the chain: batches 1 and 2. The greedy rule walks through the bins in order. Bin 1 takes batch 1. Bin 2 is crossed with bin 1, so it takes batch 2. Bin 3 is crossed with bin 2 only, among the bins already numbered, so it takes batch 1 again, and so on: along the chain the bins alternate.

- The evaluations of batch 1 move bins $1, 3, 5, \ldots$ They fill $\partial C^{11}/\partial b_{1,1}$, $\partial C^{33}/\partial b_{1,3}$, $\ldots$, and the cross spectra C^{12}, C^{23}, C^{34} through their odd end: $\partial C^{12}/\partial b_{1,1}$, $\partial C^{23}/\partial b_{1,3}$, $\partial C^{34}/\partial b_{1,3}$.

- The evaluations of batch 2 move bins $2, 4, 6, \ldots$ and fill the even ends: $\partial C^{12}/\partial b_{1,2}$, $\partial C^{23}/\partial b_{1,2}$, $\partial C^{34}/\partial b_{1,4}$, and the even auto spectra.

This is the five-bin example of Table 2, continued through every group: the alternating circles and squares of Figure 3a.

The overlap: why batches 3 and 4 appear. LRG3 and ELG1 are two different galaxy populations, luminous red galaxies and emission-line galaxies, observed in the same volume, at $z \approx 0.8$–1.1 (Table 3). An LRG3 bin and an ELG1 bin at the same redshift trace the same matter, so their cross spectrum is measured, roughly $b_{1,i}\, b_{1,j}\, P$ with P the matter power

spectrum. It carries no shot-noise term, because the two bins hold different galaxies: the $\mathrm{SN}_{0,i}$ cross terms exist only within one sample.

These pairs obey the same rule: the two bins are neighbors in redshift. They only look far apart because the bin numbers run group by group. LRG3 bin 44 ($\bar{z} = 0.932$) and ELG1 bin 55 ($\bar{z} = 0.931$) are 11 apart in number but at the same redshift, and their cross spectrum is measured. In Figure 2, drawn against redshift, the lines between the LRG and ELG rows are short.

For batching this matters because an ELG1 bin's redshift neighbors are its ELG neighbors and up to three LRG bins at nearly the same redshift. Bin 51, for example, is crossed with bins 50 and 52 (ELG1) and with bins 39, 40 and 41 (LRG3). Bins 39, 40 and 41 already sit in batches 1, 2 and 1, and bin 50 in batch 3, so bin 51 takes batch 4. The greedy rule puts the eleven ELG1 bins 50 to 60 in batches 3 (bins $50, 52, \ldots, 60$) and 4 (bins $51, 53, \ldots, 59$); batch 1 holds 33 bins and batch 2 holds 32, and the last ELG1 bin, 61, lands in batch 2. Batches 3 and 4 appear only in the overlap (Figure 3b).

Four batches, whatever the number of bins. Every bin's partners are local in redshift, so the number of batches does not grow with N. The other layouts follow the same pattern, neighbors along the chain plus overlap pairs: $\Delta z = 0.02$ has 133 cross spectra (88 along the chain) and 1 to 5 partners per bin; $\Delta z = 0.05$ has 52 (34) and 1 to 5; $\Delta z = 0.2$ has 13 (8) and 1 to 4. All four layouts get four batches.

A rule of thumb explains why the number stays small: if every bin's partners lie within k places of it in redshift order, $k + 1$ batches suffice, because going through the bins in that order, each bin has at most k numbered partners to avoid. The partners reach 3 places away on every layout. Four batches is also the minimum on every layout: each contains four bins all crossed with each other, two LRG3 and two ELG1 bins at the same redshift (bins 40, 41, 51 and 52 on $\Delta d_{\mathrm{com}} = 62$ Mpc; 36, 37, 51 and 52 on $\Delta z = 0.02$; 15, 16, 21 and 22 on $\Delta z = 0.05$; 5 to 8 on $\Delta z = 0.2$, Figure 2), and four bins all crossed with each other need four different batches.

On every layout batches 3 and 4 hold only ELG1 bins. On $\Delta d_{\mathrm{com}} = 62$ Mpc they are 11 of the 76 bins, about 15%, yet they take half of each family's evaluations.

What batching buys. The cost now follows from the number of batches, not from N. On $\Delta d_{\mathrm{com}} = 62$ Mpc:

1. one family takes 4 batches $\times$ 4 evaluations = 16 evaluations and fills all 76 of its columns, instead of $4 \times 76 = 304$;

2. the four stenciled families take $4 \times 16 = 64$ evaluations, instead of 1,216.

The same 64 hold on every layout, from 10 bins to 90 (Figure 3d).

Why not exact derivatives by hand? If each cross spectrum were a fixed bilinear expression in the biases, its derivatives could be written down exactly. The low-rank non-Limber engine of CosmoLike does not compute it that way. For each bin it approximates $\sqrt{P_{g,i}(k,z)}$, with $P_{g,i}$ the one-loop galaxy power spectrum of bin i, by a short sum of products of a function of k and a function of z: a truncated singular-value decomposition, which makes the projection fast.

It builds a cross spectrum from the geometric mean $\sqrt{P_{g,i}}\sqrt{P_{g,j}}$ of the two bins' spectra. Before the square root it floors $P_{g,i}$ at $10^{-15}\, b_{1,i}^2 P_\delta$, with P_δ the matter power spectrum, so the root always exists. The square root, the truncation and the floor make the spectra not exactly bilinear in the biases. Finite differences through the full likelihood differentiate the model exactly as it is computed.

5.3 How the steps reach CosmoLike: the wrapped setter

The obstacle: no per-bin handle among the sampled parameters. A batch evaluation needs an ordinary likelihood call in which one family is shifted in the bins of one batch: $b_{2,i} + o\,\epsilon$ in every bin i of the batch, say, with o a stencil offset and ϵ the step, and everything else fixed. The sampled parameters cannot express that. The parameter files tie each bin's linear bias to its group amplitude, $b_{1,i} = B_g/D(\bar{z}_i)$, and the likelihood computes $b_{2,i}$ and $b_{s^2,i}$ from $b_{1,i}$ through the calibrations: no sampled parameter moves one bin's $b_{2,i}$ and nothing else. Writing shifted values into CosmoLike beforehand fails too, because the likelihood recomputes and resends every per-bin value at every evaluation, overwriting whatever was written.

The completed per-bin values exist in exactly one place: the argument list of the setter, the function through which the likelihood hands them to CosmoLike's compiled code at every evaluation. So the notebook intercepts that call.

What "intercepting" means. The Fisher module exposes CosmoLike's compiled interface as the Python module `fm.ci`. Its attribute `set_nuisance_bias_tidal` is a name, not the function itself: `interface/interface.cpp` binds that name to the compiled C++ function `cosmolike_interface::set_nuisance_bias_fastpt_tidal`, and Python looks the name up anew at every call. Rebinding the name to a Python function, `fm.ci.set_nuisance_bias_tidal = tidal`, therefore redirects every caller to `tidal` – the likelihood included, which knows nothing of the notebook. The wrapper keeps a reference to the original compiled function, edits the arrays it receives, and passes them on. Nothing in the likelihood or in the C code changes.

The wrapper. `model_a_derivatives` of the base notebook, `EXAMPLE_EVALUATE_FISHER_MODEL_A.ipynb` (the file name is historical; it is the notebook that computes and caches the base), defines the wrapper next to a saved reference to the original:

```
original = fm.ci.set_nuisance_bias_tidal
seen = {}
shift = {'op': None}

def tidal(**kw):
    kw = {k: np.array(v, dtype=float) for k, v in kw.items()}
    if shift['op'] is None:
        seen.update({k: v.copy() for k, v in kw.items()})
    else:
        op, members, step = shift['op'], shift['members'], shift['step']
        if op == 'b1':
            kw['B1'][members] *= 1.0 + step
        elif op == 'b2':
            kw['B2'][members] += step
        elif op == 'bs2':
            kw['BS2'][members] += step
```

```
        elif op == 'a0':
            # alpha_0 k^2 P = 2 b1 bK k^2 P with k in H0/c units
            kw['BK'][members] += step/(2.0*kw['B1'][members]*C_OVER_H0**2)
    return original(**kw)
```

Reading it:

- The keyword arguments are the per-bin arrays the likelihood computed for this evaluation: `B1`, `B2` and `BS2` hold $b_{1,i}$, $b_{2,i}$ and $b_{s^2,i}$, and `BK` the nonlocal bias $b_{K,i}$, one entry per bin.
- `shift` is the channel between the loop and the wrapper: a small dictionary the loop fills before each evaluation. With no shift active (`shift['op'] is None`) the wrapper changes nothing and only records the fiducial arrays in `seen` – this is how the notebook learns the fiducial $b_{1,i}$, $b_{2,i}$ and $b_{s^2,i}$ without recomputing the calibrations.
- In a batch evaluation, `members` holds the bins of the batch and `step` the signed step $o\,\epsilon$. The $b_{1,i}$ step is relative (multiply by $1 + o\,\epsilon$), the $b_{2,i}$ and $b_{s^2,i}$ steps absolute. An $\alpha_{0,i}$ step enters `BK`, because CosmoLike writes the counterterm of Eq. (3) as $2\,b_{1,i}b_{K,i}\,k^2P_{\rm mm}$ with k in units of H_0/c: a step $o\,\epsilon$ of $\alpha_{0,i}$ is $b_{K,i} \to b_{K,i} + o\,\epsilon/[2\,b_{1,i}(c/H_0)^2]$ (`C_OVER_H0` $= c/H_0$ in Mpc/h).
- The last line calls the saved compiled setter with the edited arrays: CosmoLike receives a complete, consistent set of per-bin values, exactly as in an ordinary evaluation.

Install and restore. The rebinding is wrapped in `try`/`finally`:

```
fm.ci.set_nuisance_bias_tidal = tidal
try:
    m0 = model.dv()
    # ... the batch loop below, then D = model.derivatives(h=H_MAIN) ...
finally:
    fm.ci.set_nuisance_bias_tidal = original
```

A `finally` clause runs whether its block ends normally or by an exception, so no error can leave the wrapper installed: after `model_a_derivatives` returns or fails, the name points at the compiled function again, and every later evaluation in the same session is an ordinary one.

The batch loop. Inside the `try` block, the loop walks the four families and the four batches (trimmed; `OPERATOR_STEPS` holds the steps ϵ of Section 5.2: 0.01 for $b_{1,i}$, 0.02 for $b_{2,i}$ and $b_{s^2,i}$, $5\,h^{-2}\,\mathrm{Mpc}^2$ for $\alpha_{0,i}$):

```
for op, step in OPERATOR_STEPS.items():
    Dop = np.zeros((len(m0), nbins))
    for c in range(max(color) + 1):
        members = np.array([i for i in range(nbins) if color[i] == c])
        vectors = []
        for offset in fm.FIVE_POINT_OFFSETS:
            shift.update(op=op, members=members, step=offset*step)
            vectors.append(model.dv())
        shift['op'] = None
        d = fm.five_point(vectors, step)
```

```
        for i in members:
            # the b1 step is relative: (b1 dm/db1)/b1 = dm/db1
            Dop[rows[i], i] = d[rows[i]]/(b1[i] if op == 'b1' else 1.0)
    ops[op] = Dop
```

Reading it:

- `color[i]` is bin i's batch number from the greedy rule of `pair_coloring` (the paragraph "What a batch is, as a picture"): the code keeps the mathematicians' word for a batch.
- `fm.FIVE_POINT_OFFSETS` $= (2, 1, -1, -2)$ are the offsets o of the stencil; each `model.dv()` is one full data vector with the shift active, and `fm.five_point` combines the four with the weights of Eq. (14) and divides by the step.
- `rows[i]`, built once by `spectrum_rows`, lists the data-vector entries of every spectrum that contains bin i: its auto spectrum and its measured crosses. Column i receives the derivative on those rows and stays exactly zero elsewhere – the zeros of Table 2 are never computed.
- The $b_{1,i}$ column is divided by $b_{1,i}$: the relative step of size $\epsilon\, b_{1,i}$ makes the stencil return $b_{1,i}\, \partial m/\partial b_{1,i}$, as in Section 5.2.

The first cache in the way: cobaya's. Seen from cobaya (the framework that runs the likelihood), every batch evaluation asks for the same sampled point: the shift rides inside the setter, invisible to the sampler layer. Cobaya stores the result of the previous point and returns it without calling the likelihood when the point repeats – the right behavior in a chain, and exactly wrong here, where consecutive evaluations differ only inside the wrapper. `ModelFisher.dv` (`interface/cosmolike_lssty1xdesi_fisher.py`) therefore calls `self.model.loglike(point, ..., cached=False)`: the likelihood, and with it the wrapped setter, runs at every single evaluation.

The second caching layer: CosmoLike's own, one key per sector. CosmoLike keeps caches of its own, because its projections are expensive and a sampler changes only a few inputs per step. The design, shared by the whole code (`cosmolike_core`'s `README.md`, section "The sector-ladder cache check"), has three parts:

1. The inputs are grouped into sectors: the cosmology, the galaxy biases, the lens and source redshift distributions, the photo-z and intrinsic-alignment nuisances, the accuracy tables.
2. Each sector owns one key, a number. A setter compares every incoming value against the stored one (`fdiff`) and, when any differs, stores the new values and draws a fresh random number as the sector's key. For the biases (`generic_interface.cpp`): `set_nuisance_bias_fastpt_tidal` runs its four stages – linear, nonlinear-with-tidal, magnification and nonlocal bias – and any stage that finds a change executes `nuisance.random_galaxy_bias = RandomNumber::get_instance().get()`. `set_nuisance_bias_alpha2` bumps the same key; the cosmology setters bump `cosmology.random`, and `Ntable.random` belongs to the accuracy settings.

3. Every cached result stores the keys it was built under and compares them at every call. In `cosmo2D.c`, for example, a cached clustering projection recomputes when any stored key – `cosmology.random`, `Ntable.random`, `redshift.random_clustering`, `nuisance.random_galaxy_bias` and the rest – differs from the current one, and is returned as is otherwise.

Why a wrapped step can never leave an out-of-date spectrum. The wrapper changes the arrays *before* the compiled setter sees them, so the comparison of step 2 is always against the values of the previous evaluation, whoever wrote them:

- A batch evaluation shifts one family in a few bins. The setter finds the changed entries and draws a new bias key, so every cached result that depends on the galaxy bias is out of date and is recomputed. Everything keyed only on other sectors – CAMB's power spectra, the distances, CosmoLike's power tables and FAST-PT tables – keeps its keys and is reused. This is the economy of the scheme: on $\Delta d_{\rm com} = 62$ Mpc at the final settings, an evaluation whose only change is the biases takes about 36 s, against about 2.5 minutes for an evaluation at a new cosmology, which rebuilds everything (the laptop of Section 5.5).
- The return to the fiducial is itself a change: the next evaluation's arrays differ from the shifted ones, the key is drawn again, and the bias-dependent results are rebuilt once more. An out-of-date spectrum cannot survive in either direction, and the wrapper needs no cleanup beyond restoring the name: the likelihood resends every per-bin value anyway.
- One stage recomputes regardless: the low-rank factorization of Section 5.4 rebuilds its kernels and transforms at every call, and its *caller* caches the output under the sector keys (`cosmo2D.c`, `C_gg_lowrank_decomp_work`). Fresh contents under a fresh key, so the same argument applies.

One wrapped evaluation, end to end.

1. `model.dv()` calls the likelihood at the unchanged sampled point, with cobaya's cache disabled.
2. The likelihood computes the per-bin arrays from the sampled parameters and the calibrations and calls `ci.set_nuisance_bias_tidal` – now the wrapper.
3. The wrapper shifts one family's entries in the batch's bins and calls the compiled setter.
4. The setter compares entry by entry; the changed entries make it draw a new bias key, `nuisance.random_galaxy_bias`.
5. Each cached projection compares its stored keys: the bias-dependent ones rebuild, the cosmology-keyed ones are reused.
6. The data vector returns: one of the four stencil evaluations of Eq. (14).

The same trick for $\alpha_{2,i}$. The model B notebook wraps the per-bin α_2 setter, `set_nuisance_bias_alpha2`, in exactly the same way (its constant `ALPHA2_SETTER` names the setter); the term is linear, so a central difference on each batch gives the exact columns, whatever the step (Section 5.4).

What tests this, and what does not. Two kinds of checks guard the wrapped route.

The sector-key machinery itself is tested by the sector-ladder cache check, `tests/data_vector/test_cache_consistency.py` (`tests/data_vector/README.md`, section "The sector-ladder cache check"): it walks a deterministic ladder of mixed cosmology, bias and Fingers-of-God updates on all four layouts, evaluating after every step, then scrambles every sector at once and returns; the pipeline must reproduce the recorded vector bit for bit, and a second model walking the mirrored ladder must land on the same vector. One honest limitation: that test runs the frozen linear-bias configuration, so it exercises the linear setter `set_nuisance_bias`, not the one-loop `set_nuisance_bias_tidal` or `set_nuisance_bias_alpha2`.

The interception route is validated end to end, against derivatives that never pass through a wrapper:

- In the linear notebook, the per-bin $b_{1,i}$ columns, taken through a wrapper of the linear setter and summed over each group, reproduce the group-amplitude columns of the ordinary sampled-parameter stencils to at most 6.0×10^{-13} of the group column's largest entry over the four layouts; the base notebook repeats the check through the chain rule of Eq. (13), to 1.5×10^{-9}.
- The route-agreement study of Section 8 unties every per-bin parameter, samples it at its fiducial value and steps it through the full likelihood, as a chain would, with no wrapped setter. The $b_{2,i}$, $b_{s^2,i}$ and $\alpha_{0,i}$ columns of the two routes agree exactly, and the $b_{1,i}$ columns, connected through the chain rule, to 5.2×10^{-11} (the file `tests/studies/results/fisher_derivatives_agreement_dz0p05.json`).

TODO. The sector-ladder test has no one-loop rung: a ladder whose bias updates run through `set_nuisance_bias_tidal` and `set_nuisance_bias_alpha2` (model B's offsets and its $\alpha_{0,i}$, $\alpha_{2,i}$ and $\mathrm{SN}_{0,i}$ steps) is still to be added.

5.4 Exact templates

Linear terms need no stencil. Suppose the data vector depends on an amplitude A only through one added term,

$$m(\theta, A) \;=\; m_{\mathrm{rest}}(\theta) \;+\; A\,T(\theta). \tag{16}$$

Then $\partial m/\partial A = T(\theta)$ exactly. No step enters, so there is no truncation error at all, not even the ϵ^4 error of the five-point stencil, Eq. (14). T, the term per unit amplitude, is called the *template* of A: one evaluation of the term, or the term computed directly from the fiducial kernels, gives the exact column. Three families enter m this way:

- the white noise beyond Poisson $\mathrm{SN}_{0,i}$, in units of the bin's Poisson power $1/\bar{n}_i$ ($\bar{n}_i$ its tracer density);

term	route	depends on	linear in its amplitude?
density–density	low-rank (SVD, floor)	$b_{1,i}$, $b_{2,i}$, $b_{s^2,i}$, $\alpha_{0,i}$; θ	no: stencils
redshift-space rows	FKEM split	$b_{1,i}$; f, D	no amplitude of its own
α_2 counterterm	FKEM rows	$\alpha_{2,i}$; $b_{1,i}$, f	yes: central difference
$\mathrm{SN}_{0,i}$, $\mathrm{SN}_{2,i}$	likelihood templates	$\mathrm{SN}_{0,i}$, $\mathrm{SN}_{2,i}$; W_i	yes: computed directly

Table 4: The terms of a DESI band power, Eqs. (17) and (23): the route each takes, what it depends on, and whether it is exactly linear in an amplitude of its own. θ stands for the cosmological parameters and the velocity dispersions σ_v.

- the stochastic term $\mathrm{SN}_{2,i}\, k^2\mu^2$, with μ the cosine of the angle to the line of sight, in units of $f_{\mathrm{sat}}\sigma_{\mathrm{sat}}^2/\bar{n}_i$ (f_{sat} the satellite fraction, σ_{sat} the satellites' velocity dispersion in distance units);
- the radial counterterm $\alpha_{2,i}$, in $h^{-2}\,\mathrm{Mpc}^2$, which enters the counterterm density $\frac{1}{2}(b_{1,i}\alpha_{0,i} + f\mu^2\alpha_{2,i})\, k^2\delta_{\mathrm{L}}$ (f the linear growth rate, δ_{L} the linear density contrast).

That these three are *exactly* linear is not obvious: the density–density part of every DESI spectrum goes through a singular value decomposition and a square root ("Why not exact derivatives by hand?"), and a square root is linear in nothing. The resolution is that none of the three families ever enters that route. To see it, write the whole spectrum once, term by term.

The whole C_ℓ, term by term. The pipeline evaluates the cross spectrum of bins i and j as a sum of three terms, each projected by its own route, and the likelihood adds the stochastic terms afterwards:

$$C_\ell^{ij} = \underbrace{C_\ell^{\mathrm{DD},\,ij}}_{\substack{\text{low-rank route:}\\ \text{SVD, geometric mean, floor}}} + \underbrace{C_\ell^{\mathrm{RSD},\,ij}}_{\substack{\text{FKEM split:}\\ \text{density}\times\text{RSD, RSD}\times\text{RSD}}} + \underbrace{C_\ell^{\mathrm{CT},\,ij}}_{\substack{\text{FKEM rows:}\\ \text{linear in } \alpha_2}} . \tag{17}$$

Every DESI row of the data vector is the average of C_ℓ^{ij} over one band of multipoles, and averaging is linear, so every linearity below survives into the band powers. Lensing magnification would be a fourth term; this forecast sets its amplitude to zero in every DESI bin. Table 4 summarizes the terms; the paragraphs below take them one at a time.

Term 1: density–density, the only term through the SVD. C_ℓ^{DD} projects the real-space galaxy power $P_{gg,i}$ of Eq. (3). Per bin, the low-rank engine factorizes $\sqrt{P_{g,i} - P_{\infty,i}}$ by a singular value decomposition into five products of a function of k and a function of z; here $P_{\infty,i} = -(\sigma_4/2)\, D^4\, (b_{2,i} + 2b_{s^2,i}/3)^2$ is the white-noise constant that the loop subtraction leaves at high k (σ_4 a fixed integral of the linear power), and a constant power projects exactly in Limber, so it is carried there instead. A cross pair uses the geometric mean $\sqrt{P_{g,i}}\sqrt{P_{g,j}}$, and a floor guards the square root. This term depends on $b_{1,i}$, $b_{2,i}$, $b_{s^2,i}$ and $\alpha_{0,i}$ – and, through the square root, the rank truncation and the floor, on none of them exactly linearly.

Term 2: the redshift-space rows, through the FKEM split. With $W_i(\chi)$ the bin's kernel – its redshift distribution as a function of comoving distance χ, after the Fingers-of-God

smoothing – D the growth factor, f the growth rate and j_ℓ the spherical Bessel function, the bin's linear redshift-space kernel and its RSD part are

$$\Delta^i_\ell(k) \;=\; \int d\chi\, W_i\, D\,\big[\, b_{1,i}\, j_\ell(k\chi) - f\, j''_\ell(k\chi)\,\big], \qquad R^i_\ell(k) \;=\; -\int d\chi\, W_i\, D\, f\, j''_\ell(k\chi) \tag{18}$$

(each factor $f\mu^2$ of the flat-sky power becomes a $-j''_\ell$ in the exact projection; the summary note, `notes/bias_model_summary`, paragraph "Non-Limber spectra"). Writing $\Delta^{i,\mathrm{D}}_\ell$ for the density part alone, the $b_{1,i}\, j_\ell$ term of Δ^i_ℓ, the term is

$$C_\ell^{\mathrm{RSD},\,ij} \;=\; \frac{2}{\pi}\int dk\, k^2\, P_{\mathrm{lin}}(k)\,\Big[\,\Delta^i_\ell\,\Delta^j_\ell - \Delta^{i,\mathrm{D}}_\ell\,\Delta^{j,\mathrm{D}}_\ell\,\Big] \;+\; \text{Limber corrections on the same rows,} \tag{19}$$

the density×RSD and RSD×RSD products, computed with the separable linear spectrum by FFTLog and corrected in Limber with the full one (the FKEM split). The term depends on $b_{1,i}$, already covered by its stencil, and on cosmology through f and D. It has no amplitude of its own.

Term 3: the α_2 counterterm, linear by construction. In the convention of the DESI DR1 full-shape analysis [2], the $\alpha_{2,i}$ part of the counterterm density is $\frac{1}{2} f\mu^2 \alpha_{2,i}\, k^2 \delta_{\mathrm{L}}$, and the power it adds is *twice its cross* with the linear galaxy density $(b_{1,i} + f\mu^2)\,\delta_{\mathrm{L}}$:

$$P_{\mathrm{CT}} \;=\; (b_{1,i} + f\mu^2)\, f\, \alpha_{2,i}\, \mu^2 k^2\, P_{\mathrm{lin}}(k, z). \tag{20}$$

The counterterm's power with itself, which would carry $\alpha^2_{2,i}$, is of higher order and is dropped by the convention. In the exact projection the counterterm field of bin i has the transfer $(\alpha_{2,i}/2)\, k^2 R^i_\ell$, and CosmoLike adds (`cosmo2D.c`, `has_alpha2_galaxies`; `galpha2` in `bias.c` stores the per-bin value)

$$C_\ell^{\mathrm{CT},\,ij} \;=\; \frac{2}{\pi}\int dk\, k^2\, \big[k^2 P_{\mathrm{lin}}(k)\big]\,\Big[\frac{\alpha_{2,i}}{2}\, R^i_\ell\, \Delta^j_\ell + \frac{\alpha_{2,j}}{2}\, \Delta^i_\ell\, R^j_\ell\Big]\,. \tag{21}$$

Every occurrence of $\alpha_{2,i}$ in the code multiplies a product of one counterterm transfer and one linear transfer; nowhere is it squared. The term contains no density×density piece, so it never touches the SVD: it rides the FKEM rows, which enter Eq. (21) linearly, and in the low-rank path it is carried unchanged by the FKEM pass and the cross-pair pass. The split uses P_{lin} in the FFTLog sum and in both of its Limber pieces, so it cancels for this term at high ℓ. The projected band powers are therefore exactly linear in $\alpha_{2,i}$.

Its columns come from its own per-bin setter, `set_nuisance_bias_alpha2`, wrapped as in Section 5.3: a central difference on each batch, two evaluations per batch and eight in all, exact because the term is linear. The model B notebook verifies the linearity: the forward and backward differences agree to at most 1.6×10^{-10} of the column (4.3×10^{-11} on the three thinner layouts).

Terms 4 and 5: the stochastic templates, added outside CosmoLike. $\mathrm{SN}_{0,i}$ adds $\mathrm{SN}_{0,i}/\bar{n}_i$ to $P_{g,i}$, a constant in k. For a constant power S the k integral of the exact projection closes by the orthogonality of the Bessel functions, $\frac{2}{\pi}\int dk\, k^2\, j_\ell(k\chi_1)\, j_\ell(k\chi_2) = \delta_{\mathrm{D}}(\chi_1 - \chi_2)/\chi_1^2$ (δ_{D} the Dirac delta), so the spectrum gains, independently of ℓ,

$$C_\ell^{ij} \;\supset\; S\int d\chi\, \frac{W_i(\chi)\, W_j(\chi)}{\chi^2}, \qquad S = \frac{\mathrm{SN}_{0,i}}{\bar{n}_i} \;\text{ on the auto spectrum.} \tag{22}$$

A cross pair inside one tracer sample takes the mean of its two bins' stochastic powers, and a pair of two different samples takes none: different galaxies share no stochastic term. The integral is fixed by the fiducial kernels, so the band powers are exactly linear in $\mathrm{SN}_{0,i}$, with the integral as the template. The $\mathrm{SN}_{2,i}$ term is $S_2\,k^2\mu^2$ with $S_2 = \mathrm{SN}_{2,i}\,f_{\mathrm{sat}}\sigma^2_{\mathrm{sat}}/\bar{n}_i$; since $k^2\mu^2 = k^2_\parallel$ is the Fourier image of a second derivative along the line of sight, two integrations by parts move one derivative onto each kernel, and the template is $\int d\chi\, W_i' W_j'/\chi^2$ with $W' = dW/d\chi$ (the correlated-prior note, section "Model A"). The likelihood computes both templates directly from the fiducial kernels (`stochastic_templates` of `likelihood/desi_cl.py`, orders 0 and 2) and adds amplitude times template to the vector CosmoLike returned, keeping the sum in `last_datavector`. The amplitudes never enter CosmoLike's projection at all, let alone the SVD.

The consequence. Collecting the terms, the DESI part of the data vector is

$$m \;=\; m_{\mathrm{DD+RSD}}\big(\{b_{1,i}\},\{b_{2,i}\},\{b_{s^2,i}\},\{\alpha_{0,i}\};\,\theta\big) + \sum_i \alpha_{2,i}\,T_i^{\alpha_2} + \sum_i \mathrm{SN}_{0,i}\,T_i^{(0)} + \sum_i \mathrm{SN}_{2,i}\,T_i^{(2)}, \tag{23}$$

with θ the cosmological parameters and the σ_v. The last three groups have the form of Eq. (16), so their columns are the templates themselves; only the first group needs stencils. One subtlety: $T_i^{\alpha_2}$ depends on $b_{1,i}$ (through Δ_ℓ^j) and on cosmology, so the $\alpha_{2,i}$ column is exact *at fixed* $b_{1,i}$ – which is precisely what the partial derivative $\partial m/\partial\alpha_{2,i}$ is.

Why $\alpha_{0,i}$ is not a template. It looks like one: a counterterm amplitude multiplying $k^2 P_{\mathrm{mm}}$ in Eq. (3). But it sits in the real-space density–density power, the one term that goes under the square root, the SVD and the geometric mean, so the projected spectra are not exactly linear in it; the same holds for $b_{1,i}$, $b_{2,i}$ and $b_{s^2,i}$. That is the dividing line of this section: the families that stay outside the density–density term get exact templates, and everything that passes through the SVD gets the stencils of Section 5.3.

The templates themselves are validated in the higher-order note, `notes/higher_order_bias`, section "Validation": the exact $\mathrm{SN}_{2,i}$ projection against a brute-force j_ℓ, j_ℓ'' double integral, and the linearity of the $\alpha_{2,i}$ and $\mathrm{SN}_{2,i}$ terms by forward against backward differences.

5.5 How many data-vector evaluations, and how long

Counted in the builders and the Fisher module, the base of one layout takes

$$N_{\mathrm{eval}} \;=\; 1 + 4\,N_{\mathrm{stencil}} + 4\,N_{\mathrm{batch}}\,N_{\mathrm{fam}} + (2\,N_{\mathrm{batch}} + 2) + 1 \tag{24}$$

evaluations of the data vector, term by term in Table 5. It is 156 on every layout, because no term grows with the number of bins.

The cost of one evaluation. Each evaluation is a complete data vector: CAMB, then CosmoLike's non-Limber projection of every thin bin. Its cost depends mostly on g, the refinement factor of the comoving-distance grid on which the non-Limber projection integrates (Section 8). Table 6 gives the times the convergence study measured on $\Delta z = 0.05$ (36 bins), on AC power. The MCMC settings are those of the example yamls used for Markov-chain Monte Carlo runs, the sampling of the full likelihood.

term	what it is	evaluations
1	the fiducial vector	1
$4\,N_{\mathrm{stencil}}$	20 stencil parameters: 7 cosmology, 7 σ_v, 6 B_g	80
$4\,N_{\mathrm{batch}}\,N_{\mathrm{fam}}$	4 batches, 4 families $\{b_{1,i}\}$, $\{b_{2,i}\}$, $\{b_{s^2,i}\}$, $\{\alpha_{0,i}\}$	64
$2\,N_{\mathrm{batch}} + 2$	$\{\alpha_{2,i}\}$: 2 per batch, plus a fiducial vector before and after	10
1	the fiducial run of the $\{\mathrm{SN}_{2,i}\}$ templates	1
	total	156

Table 5: The evaluations behind one layout's base. The $\{\mathrm{SN}_{0,i}\}$ and $\{\mathrm{SN}_{2,i}\}$ templates cost no evaluation beyond a fiducial one.

configuration	grid factor g	seconds per evaluation
MCMC settings	32	3.9
starting configuration, grid lowered	64	11.4
starting configuration	128	17.0
starting configuration, grid raised	256 (final)	28.0
starting configuration, grid raised	512	48.4
final settings	256 (final)	19.5

Table 6: The cost of one data-vector evaluation on $\Delta z = 0.05$, measured by the convergence study on a MacBook Pro with the Apple M2 Pro chip (8 OpenMP threads, one BLAS thread), on AC power with no other heavy job running, except the $g = 256$ run of the starting configuration, which started while one other job was running. The final settings ran 130 evaluations in 42 minutes. Section 8 describes the configurations.

The total time. The base of a layout takes $N_{\mathrm{eval}} \times t_{\mathrm{eval}}$, with t_{eval} the time of one evaluation, measured on a MacBook Pro with the Apple M2 Pro chip (8 OpenMP threads, one BLAS thread, on AC power with no other heavy job). At the final settings:

- $\Delta z = 0.05$, from the measured t_{eval}: 156×19.5 s ≈ 51 minutes.
- The other layouts, a rough estimate that scales t_{eval} with the number of bins: $\Delta d_{\mathrm{com}} = 62$ Mpc (76 bins) 156×41 s ≈ 1.8 hours; $\Delta z = 0.02$ (90 bins) 156×49 s ≈ 2.1 hours; $\Delta z = 0.2$ (10 bins) 156×5.4 s ≈ 14 minutes.

The four layouts then take about five hours. On other hardware – an Intel workstation, an x86 cluster node – the time per evaluation differs, so these numbers are a guide to the order of magnitude, not a benchmark.

6 A scale cut is a set of rows

A scale cut removes the band powers that small, nonlinear scales dominate. A cut at k_{max}, quoted at $z = 0$, cuts bin i at $k_{\mathrm{max}}/D(\bar{z}_i)$, by the non-Limber ultraviolet-fraction criterion of Saraivanov et al. [3] within the shipped mask: a band is kept while less than 5% of it comes from the power spectrum above that wavenumber. The Fisher matrix at a cut follows in four steps.

1. The cut keeps a set K_m of rows of m. It drops the other rows of B and keeps every column: a cut never changes a derivative. B_K is the kept rows of B, and C_{KK} the kept block of the covariance.

2. The data part of the Fisher matrix of a model at this cut is $F_{\text{data}} = \mathsf{D}_K^{\mathsf{T}} f_A \mathsf{C}_{KK}^{-1} \mathsf{D}_K$, with $f_A = 12{,}300/14{,}000$ the factor that rescales the inverse covariance to the survey area (Section 2). Since $\mathsf{D}_K = \mathsf{B}_K\mathsf{M}$,

$$F_{\text{data}} \;=\; \mathsf{M}^{\mathsf{T}}\big(f_A\,\mathsf{B}_K^{\mathsf{T}}\,\mathsf{C}_{KK}^{-1}\,\mathsf{B}_K\big)\mathsf{M} \;=\; \mathsf{M}^{\mathsf{T}}\,\mathsf{G}_m\,\mathsf{M}, \qquad \mathsf{G}_m \;\equiv\; f_A\,\mathsf{B}_K^{\mathsf{T}}\,\mathsf{C}_{KK}^{-1}\,\mathsf{B}_K. \tag{25}$$

   G_m is what the data of cut m say about every pair of knobs: its entry (a, b) is the information shared by knobs a and b.

3. G_m contains neither M nor a prior. It depends only on the base, the cut and the covariance, so the notebooks compute it once per cut, and every model and prior at that cut reuses it.

4. The priors add their precision: $F = \mathsf{M}^{\mathsf{T}}\mathsf{G}_m\mathsf{M} + F_{\text{prior}}$.

The toy, cut. Suppose cut m drops the cross spectrum C^{23}, which fails the ultraviolet criterion. Then:

- $K = \{C^{11}, C^{22}, C^{33}, C^{12}\}$, and B_K is the 4×7 matrix of the first four rows of Eq. (7);
- C_{KK} is 4×4, and G_m is 7×7;
- the group model's F is $(6 \times 7)(7 \times 7)(7 \times 6) = 6 \times 6$, and the free model's 7×7, both from the same G_m.

After this cut, v_3 survives only on C^{33}.

The real sizes. On $\Delta d_{\text{com}} = 62$ Mpc, B is $10{,}494 \times 547$. The shipped mask keeps 5,674 rows, so B_K is $5{,}674 \times 547$, C_{KK} is $5{,}674 \times 5{,}674$ and G_m is 547×547. F is 150×150 for the paper model. The large block is handled once per cut, and every Fisher system at that cut (one Fisher matrix with its inverse and results, for each model and prior the notebooks evaluate) reuses G_m.

How G_m is computed. The notebooks compute G_m in `layout_grams`, without ever inverting the covariance:

1. they write $\mathsf{C}_{KK} = LL^{\mathsf{T}}$ with L lower triangular, the Cholesky factorization, a matrix analogue of the square root;
2. they solve $L\,W = \mathsf{B}_K$ for W row by row, which a triangular L allows (forward substitution);
3. they form $\mathsf{G}_m = f_A\,W^{\mathsf{T}}W$, with f_A the area rescaling of the inverse covariance.

This saves work, avoids amplifying round-off when the covariance is ill-conditioned (when its inverse magnifies small errors), and makes G_m symmetric with no negative information, by construction.

Each layout gets eight such matrices: the seven cuts $k_{\max} = 0.05, 0.075, \ldots, 0.2\,h/\mathrm{Mpc}$, and the shipped mask alone, the comparison point of the information test below. The base notebook, which works at one cut, forms $\mathsf{D} = \mathsf{BM}$ explicitly and multiplies it by the inverse of the kept covariance block; the two routes agree (Section 9).

The priors. F_{prior} adds precision and needs no data. The diagonal Gaussian priors, all centered on zero, sit on σ_v (from the example yaml) and on the model's group parameters. The counterterm widths, $0.5/k_{\max}^2$, change with the cut, so the diagonal is rebuilt for each cut (`prior_diagonal`).

The linear-bias prior acts on the block of the β_i. The baseline and pessimistic priors add, group by group, the penalty of their memory model on bin-to-bin wiggles of the linear biases (`group_bias_precision`; the correlated-prior note, sections "What the prior charges" and "How the prior enters the Fisher matrix"). The free prior adds nothing, and the growth-tied prior is a change of M.

Parameters without data. At small cuts some bins keep no band, and their columns vanish on K_m. A parameter whose information at the cut, data plus prior, falls below 10^{-10} of its information on the shipped mask with the same priors leaves the fit: it is removed from M and from F, and the quoted FoMs, errors and shifts are over the remaining parameters. A Gaussian prior keeps a parameter in. A cut that keeps no band (as $\Delta z = 0.02$ at $k_{\max} = 0.05\,h/\mathrm{Mpc}$) or leaves a cosmological parameter without data gives no fit.

The pseudo-inverse. A plain inverse fails in one more case. On $\Delta d_{\mathrm{com}} = 62$ Mpc at $k_{\max} = 0.075\,h/\mathrm{Mpc}$ only 7 of the 18 BGS bins keep a band. A flat-prior BGS tilt can then lose every band that carries information on it, leaving one combination of parameters with no information at all. Along such a direction a plain inverse amplifies round-off by about 10^{16} and can turn it into a spurious shift of sigmas.

`stable_inverse` therefore takes a pseudo-inverse, an inverse that skips the directions with no information:

1. it scales F to unit diagonal, $\tilde{F} = SFS$ with $S = \mathrm{diag}(F)^{-1/2}$, so that the parameters' units do not matter;

2. it diagonalizes $\tilde{F} = V \Lambda V^{\mathsf{T}}$: the columns of V are the parameter combinations measured independently, and the eigenvalues in Λ the information in each;

3. it drops the directions whose eigenvalue is below 10^{-12} of the largest, and returns

$$F^{-1} \;=\; S\, V_\kappa\, \Lambda_\kappa^{-1}\, V_\kappa^{\mathsf{T}}\, S, \tag{26}$$

   with κ the kept directions.

A dropped direction must leave (w_0, w_a), Ω_m and A_s alone: the scan gives no fit when one touches them by more than 10^{-6}. The case above, the flat BGS tilts on $\Delta d_{\mathrm{com}} = 62$ Mpc at $k_{\max} = 0.075\,h/\mathrm{Mpc}$, is the mechanism. Across the model B notebook's scan, 20 of its 336

Fisher systems drop a direction, each touching the scored parameters by at most 7.2×10^{-10}; the systems of the convergence study drop none. The untied comparison of the summary note drops more. Its version with correlated residuals loses the residual trends that duplicate the flat group tilts: 24 directions at most cuts on the three thinner layouts, each touching the scored parameters by less than 10^{-8}. On $\Delta d_{\rm com} = 62$ Mpc at $k_{\rm max} = 0.075\,h/\mathrm{Mpc}$ that version gives no fit.

7 Everything downstream reuses G

Once G_m is built, nothing in the notebooks touches an $n \times n$ matrix or the data vector again. F is Eq. (25) plus the priors, inverted by Eq. (26), and σ_j and the FoMs follow as in Section 2.

Shifts. A truth is an assumed true universe that departs from the fitted model, for example one with $\alpha_{2,i} \neq 0$ when the fit has no $\alpha_{2,i}$. It moves the data by δm. When δm lies in the span of the base columns, $\delta m = \mathsf{B}c$, with c a vector of $n_{\rm base}$ coefficients:

- $\alpha_{2,i} = 12.5\,h^{-2}\,\mathrm{Mpc}^2$ in every bin i is $c = 12.5$ on the $\alpha_{2,i}$ block;
- an offset truth puts its values on the $b_{2,i}$ and $b_{s^2,i}$ blocks;
- a fractional drift δ_i of the linear biases is $c = \sum_i \beta_i\,\delta_i\,\mathsf{M}_{\beta_i}$, with M_{β_i} the column of Eq. (13).

To first order the best fit then moves by

$$\delta p \;=\; F^{-1}\mathsf{D}^{\mathsf{T}}\mathsf{C}^{-1}\delta m \;=\; F^{-1}\mathsf{M}^{\mathsf{T}}\,\mathsf{G}_m\,c \tag{27}$$

(the correlated-prior note, "The shift formula"). $\delta m = \mathsf{B}c$ is exact for the $\alpha_{2,i}$, $\mathrm{SN}_{0,i}$ and $\mathrm{SN}_{2,i}$ truths, whose terms are linear, and first order for truths in $b_{1,i}$, $b_{2,i}$ and $b_{s^2,i}$, since the band powers are quadratic in them. A truth costs one product with G_m.

Random truths. Eq. (27) is linear in c, so a family of random truths collapses into one matrix: for drifts δ_i of the linear biases the (w_0, w_a) shift is $\delta w = A\,\delta$, with A the (w_0, w_a) rows of Eq. (27) applied to the N columns $\beta_i \mathsf{M}_{\beta_i}$. Two thousand truths cost one matrix product.

The exact $\langle\chi^2\rangle$. Whether a fit's error bars are honest is tested by averaging $\chi^2_w = (\hat{w} - w_{\rm true})^{\mathsf{T}}\mathsf{C}_w^{-1}(\hat{w} - w_{\rm true})$, with $\hat{w}$ the fitted (w_0, w_a), over the truths and the data noise. Every source is Gaussian and enters linearly, so the average is exact in closed form (the correlated-prior note, "The test statistic"):

$$\langle\chi^2_w\rangle \;=\; \Delta^2_{\rm drift} \;+\; \mathrm{tr}\Big[\mathsf{C}_w^{-1}\left(A\,\Sigma\,A^{\mathsf{T}} + R\,\Sigma_\theta\,R^{\mathsf{T}} + N_w\right)\Big]\,. \tag{28}$$

- $\Delta_{\rm drift}$ is the (w_0, w_a) shift, in units of the ellipse, of the known drift alone: the smooth change of the linear biases with redshift that physics predicts.
- Σ is the covariance of the truth's remaining scatter of the linear biases.
- R and Σ_θ are the response to, and the prior variance of, any other parameter drawn at random ($\alpha_{2,g}$ per group in the random test of model B).

- N_w is the (w_0, w_a) block of $F^{-1}F_0F^{-1}$, the share of the data noise and the Gaussian priors, with F_0 the Fisher matrix without the linear-bias prior.

The notebooks report $\sqrt{\langle\chi^2\rangle/n_{\rm dof}}$, with $n_{\rm dof} = 2$ for (w_0, w_a), and $n_{\rm dof} = 1$ for Ω_m or A_s with that parameter's own row; 1 means honest error bars. The random draws only make the percentile bars of the correlated-prior note's figures.

The cost. Each G_m costs one Cholesky factorization of at most 7,621 rows, about $n_K^3/3 \approx 1.5 \times 10^{11}$ floating-point operations, and one triangular solve, eight times per layout. A Fisher system then costs $\mathsf{M}^\mathsf{T}\mathsf{G}_m\mathsf{M}$, with $n_{\rm base} \le 645$ and $n_{\rm par} \le 164$ (model B + tilts on $\Delta z = 0.02$), about $2\,n_{\rm base}^2 n_{\rm par} \approx 1.4 \times 10^8$ operations, plus the diagonalization of an $n_{\rm par} \times n_{\rm par}$ matrix: a fraction of a second.

The $k_{\rm max}$ scan of the model B notebook has 336 systems (four layouts, seven cuts, four variants, and the baseline, pessimistic and free priors; the growth-tied prior fails the accuracy test and is left out), 324 with a fit and 12 without any band. In operations, all of them together cost less than the Cholesky factorizations of one layout. Without the base, each variant would need its own derivatives, at the cost of Section 5.5 per layout, and each truth its own data vector.

7.1 What is precomputed, and what is computed at every run

The work splits into three levels.

1. **On disk, once per layout (hours):** the base columns and the fiducial vector $m(p_0)$. Table 7 lists every array on $\Delta d_{\rm com} = 62$ Mpc.

2. **In memory, at every run (seconds):** for each scale cut m, its kept rows K_m and the matrix $\mathsf{G}_m = f_A\,\mathsf{B}_K^\mathsf{T}\mathsf{C}_{KK}^{-1}\mathsf{B}_K$, with $f_A = 12{,}300/14{,}000$ the factor that rescales the inverse covariance to the survey area (Section 2), 547×547 on $\Delta d_{\rm com} = 62$ Mpc (Section 6). The covariance is read fresh from its file at every run.

3. **For every model and prior (a fraction of a second):** M, then $F = \mathsf{M}^\mathsf{T}\mathsf{G}_m\mathsf{M} + F_{\rm prior}$, its inverse, the FoMs, the shifts and $\langle\chi^2\rangle$ (Section 6 and this section). No evaluation of the data vector at all.

What forces the saved derivatives to be recomputed. Only what changes the knobs' responses: the fiducial point, the likelihood settings (accuracy, options, parameter files) and the code. The fingerprint of Section 8 catches changes of the input; a change of the code needs a flag set by hand (Section 10). The recipe M, the priors, the scale cut and the covariance file never force a recompute.

8 Derivative accuracy

Why a derivative needs more than a χ^2. Every evaluation of m carries numerical noise: the integration grids, the non-Limber distance grid and CAMB's tolerances change the vector by small amounts that do not vary smoothly with the parameters. A finite difference divides

array	shape	what it is
`EXAMPLE_EVALUATE_FISHER_MODEL_A_DERIVATIVES_1.pkl` (about 34 MB)		
`m0`	10,494	the fiducial band powers $m(p_0)$
`D`	$10{,}494 \times 20$	stencil columns: 7 cosmology, 7 σ_v, 6 B_g (check only)
`ops['b1']`	$10{,}494 \times 76$	column i: $\partial m/\partial b_{1,i}$, the linear bias of bin i alone
`ops['b2']`	$10{,}494 \times 76$	column i: $\partial m/\partial b_{2,i}$
`ops['bs2']`	$10{,}494 \times 76$	column i: $\partial m/\partial b_{s^2,i}$
`ops['a0']`	$10{,}494 \times 76$	column i: $\partial m/\partial \alpha_{0,i}$, per $h^{-2}\,\mathrm{Mpc}^2$
`T`	$10{,}494 \times 76$	column i: $\partial m/\partial \mathrm{SN}_{0,i}$
`b1`, `b2`, `bs2`	76 each	the fiducial $b_{1,i}$, $b_{2,i}$, $b_{s^2,i}$ of every bin
`EXAMPLE_EVALUATE_FISHER_MODEL_A_SN2_1.pkl` (about 13 MB)		
`T2`	$10{,}494 \times 76$	column i: $\partial m/\partial \mathrm{SN}_{2,i}$
`EXAMPLE_EVALUATE_FISHER_MODEL_B_TEMPLATES_1.pkl` (about 19 MB)		
`a2`	$10{,}494 \times 76$	column i: $\partial m/\partial \alpha_{2,i}$, per $h^{-2}\,\mathrm{Mpc}^2$

Table 7: The precomputed arrays of $\Delta d_{\mathrm{com}} = 62$ Mpc, in Python's pickle files: $n = 10{,}494$ band powers and $N = 76$ bins. Each $10{,}494 \times 76$ block is 6.4 MB in double precision. The files also hold the settings they were made with, and the second and third repeat `m0` and the $\partial m/\partial \mathrm{SN}_{0,i}$ columns for the checks of Section 9. In the toy's notation, `ops['b1']` is $[u_1, \ldots, u_N]$ and `ops['b2']` is $[v_1, \ldots, v_N]$.

that noise by the step, so a setting that converges a χ^2 (the MCMC criterion is $|\Delta\chi^2| < 0.2$) can leave a derivative noisy.

The noise biases the forecast one way. A column $d_j + e_j$, with e_j its noise, gives $F_{jj} = (d_j + e_j)^{\mathsf{T}}\mathsf{C}^{-1}(d_j + e_j)$, whose average exceeds the true value by $\langle e_j^{\mathsf{T}}\mathsf{C}^{-1}e_j\rangle > 0$: noise looks like information, and the FoM comes out too high.

The study: a starting configuration, and one control at a time. The convergence study, `tests/studies/run_fisher_derivatives.py`, measures the effect on $\Delta z = 0.05$ for the paper model of Section 3. It scores the marginalized errors of w_0, w_a, Ω_m and A_s, both FoMs, and the shifts of the break-test truths (truths that carry a term the fit lacks or holds with a prior; the correlated-prior note, "Model B and the reach of model A"), at $k_{\max} = 0.075$ and $0.2\,h/\mathrm{Mpc}$.

The scan starts from one configuration, the starting configuration: a grid factor $g = 128$ with every other control raised, CosmoLike's `accuracyboost` 4 and `integration_accuracy` 2, the low-rank engine's rank 8 (the number of terms kept in its sum) and multipole cap 2000, and CAMB's `AccuracyBoost` 2.0, `k_per_logint` 30 and maximum wavenumber $80\,\mathrm{Mpc}^{-1}$. The scan then varied one control at a time away from it. Here g is the product `accuracyboost` × `nonlimber_accuracyboost`, and the non-Limber projection integrates over about 511 g comoving distances; the MCMC value on $\Delta z = 0.05$ is $g = 32$. Each control was lowered toward its MCMC value; the grid was also raised to 256 and 512. The five-point stencil at step h on $g = 512$ is the converged value every result is compared with. The final tier, chosen at the end of the section, uses $g = 256$. (The study's result files name the two endpoints by key: `reference` is the starting configuration and `candidate` the final settings.)

The noise of one evaluation. The computed vector is not the smooth model alone. Write

$$m_{\mathrm{c}}(\theta) \;=\; m(\theta) \;+\; n(\theta), \tag{29}$$

with m the model, smooth in the parameters, and n the numerical error of one evaluation, from the causes named in "Why a derivative needs more than a χ^2": the integration grids, the non-Limber distance grid and CAMB's tolerances. n is small, but it does not vary smoothly with θ: a tiny move of the parameters shifts where the model falls on the fixed grids, and the error changes irregularly from point to point. The study measures the size of n from the five stencil points themselves (`stencil_noise` in `tests/studies/run_fisher_derivatives.py`), with no extra evaluations. The next paragraphs build the estimator one step at a time.

The fourth difference removes the smooth part. Take the five equally spaced points $\theta_0 + k\epsilon$, $k = -2, \ldots, 2$, and combine them with the weights $c_k = (1, -4, 6, -4, 1)$, the binomial coefficients of $(1 - x)^4$ with alternating signs:

$$\Delta^4 m \;=\; \sum_{k=-2}^{2} c_k\, m(k) \;=\; m(-2) - 4\,m(-1) + 6\,m(0) - 4\,m(1) + m(2), \tag{30}$$

writing $m(k)$ for $m_{\rm c}(\theta_0 + k\epsilon)$. Expand each point of the smooth part in a Taylor series around θ_0,

$$m(\theta_0 + k\epsilon) \;=\; \sum_{n=0}^{\infty} \frac{(k\epsilon)^n}{n!}\, m^{(n)}(\theta_0), \tag{31}$$

and collect the terms order by order: the n-th derivative enters $\Delta^4 m$ with the coefficient $(\epsilon^n/n!)\sum_k c_k\, k^n$. These sums can be checked by hand:

n	0	1	2	3	4	5	6
$\sum_k c_k\, k^n$	0	0	0	0	24	0	120

For $n = 4$ the sum is $16 - 4 + 0 - 4 + 16 = 24$; every odd order vanishes because the weights are symmetric, $c_{-k} = c_k$, while k^n is odd. So everything up to the third derivative cancels exactly, the fourth order survives with $\epsilon^4\,(24/4!) = \epsilon^4$, and the next surviving order is $n = 6$ with $120/6! = 1/6$:

$$\Delta^4 m\big|_{\rm smooth} \;=\; \epsilon^4\, m''''(\theta_0) \;+\; \frac{\epsilon^6}{6}\, m^{(6)}(\theta_0) \;+\; \ldots \tag{32}$$

The study's steps are a few percent of each parameter, and the model varies smoothly on the scale of the parameter itself, so $\epsilon^4 m''''$ is suppressed by ϵ^4, roughly seven orders of magnitude: for a smooth vector the fourth difference is essentially zero. What survives in Eq. (30) is the noise.

Independent noise adds in quadrature: the 70. The noise part of the fourth difference is $\sum_k c_k\, n_k$, with n_k the error at point k. Errors that change irregularly from point to point are modelled as independent draws with a common variance σ^2. The variance of a weighted sum of independent terms is the sum of the squared weights times σ^2:

$$\mathrm{Var}\Big(\sum_k c_k\, n_k\Big) \;=\; \sigma^2 \sum_k c_k^2 \;=\; (1 + 16 + 36 + 16 + 1)\,\sigma^2 \;=\; 70\,\sigma^2. \tag{33}$$

So $|\Delta^4 m|/\sqrt{70}$ estimates σ, the noise of one evaluation. One stencil is enough: the data vector has thousands of entries, the fourth difference is taken entry by entry, and the length of the result averages over all of them, so a single realization gives a tight estimate.

A toy, with invented numbers. Take one entry, the smooth model $m(\theta) = e^\theta$ at $\theta_0 = 0$ with $\epsilon = 0.01$, and plant noise of true rms 0.8×10^{-6} at the five points: $n_k = (1.3, -0.4, 0.9, -0.6, 0.2) \times 10^{-6}$. The smooth part of the fourth difference is $\epsilon^4 e^{\theta_0} = 1.0 \times 10^{-8}$, a thousand times below the noise combination. The noise part is $1.3 + (-4)(-0.4) + 6\,(0.9) + (-4)(-0.6) + 0.2 = 10.9$, times 10^{-6}, so the estimate is $10.9 \times 10^{-6}/\sqrt{70} = 1.3 \times 10^{-6}$: the right size, though off the true 0.8×10^{-6} by a factor 1.7, as a single entry must be, since one number is estimating a variance. Averaged over the thousands of entries of a real data vector, that scatter disappears.

Measuring a vector in the data's own units. A raw $|\Delta^4 m|$ would mix entries with very different error bars, and neighbouring band powers are correlated. The cure is to transform the vector first, so that, measured against the data's covariance C, the error of every entry is one and no two entries are correlated. After the transformation each entry counts in units of its own error bar, and the plain sum of squares of the transformed vector is a χ^2.

The transformation, through the eigenvectors. Diagonalize the covariance, $\mathsf{C} = U\Lambda U^\mathsf{T}$, with U the orthogonal matrix of its eigenvectors and Λ the diagonal matrix of the variances along them. For a vector v, the rotation $U^\mathsf{T} v$ gives its components along the eigenvectors, the directions in which the data's errors are uncorrelated, and $\Lambda^{-1/2}$ then divides each component by its standard deviation. So

$$y \;=\; \Lambda^{-1/2}\, U^\mathsf{T}\, v \tag{34}$$

has unit, uncorrelated errors, and $|y|^2 = v^\mathsf{T} U \Lambda^{-1} U^\mathsf{T} v = v^\mathsf{T} \mathsf{C}^{-1} v$. This transformation is called *whitening*, after white noise: noise with equal power in every direction, as white light holds every colour.

Why a Cholesky factor does the same job. The code whitens with the Cholesky factorization instead, $\mathsf{C} = LL^\mathsf{T}$ with L lower triangular (a matrix analogue of the square root): $y = L^{-1}v$ has covariance $L^{-1}\,\mathsf{C}\,L^{-\mathsf{T}} = 1$, so its errors are also unit and uncorrelated, and its length is the same, $|L^{-1}v|^2 = v^\mathsf{T}\mathsf{C}^{-1}v$. The two transforms differ only by a rotation, which changes no length; the Cholesky factorization is cheaper and numerically stable, so the code uses it.

A 2×2 toy. Two band powers with error bars 1 and 2 and correlation 0.5 have

$$\mathsf{C} = \begin{pmatrix} 1 & 1 \\ 1 & 4 \end{pmatrix}, \qquad L = \begin{pmatrix} 1 & 0 \\ 1 & \sqrt{3} \end{pmatrix}, \qquad L^{-1} = \begin{pmatrix} 1 & 0 \\ -1/\sqrt{3} & 1/\sqrt{3} \end{pmatrix} \tag{35}$$

(the off-diagonal entry is $0.5 \times 1 \times 2 = 1$; multiplying out LL^T recovers C). Take the difference vector $v = (1, 1)$: whitened, $y = L^{-1}v = (1, 0)$, and $|y|^2 = 1$. Directly, $\mathsf{C}^{-1} = \frac{1}{3}\left(\begin{smallmatrix} 4 & -1 \\ -1 & 1 \end{smallmatrix}\right)$ and $v^\mathsf{T}\mathsf{C}^{-1}v = (4 - 1 - 1 + 1)/3 = 1$: the same χ^2. Counting each entry against its own error bar alone would give $1/1 + 1/4 = 1.25$; the two entries are correlated, so part of the misfit repeats and whitening counts it once. The eigenvector route, with variances $(5 \pm \sqrt{13})/2 \approx 4.30$ and 0.70, gives a different y of the same length 1: the two transforms differ by a rotation.

The estimate in these units. The study whitens the fourth difference with the shipped-mask covariance at the survey area (f_A applied) and reports

$$\hat{\sigma} \;=\; \left|L^{-1}\,\Delta^4 m\right| / \sqrt{70}\,: \tag{36}$$

the numerical error of one evaluation, measured in the data's own error bars ($\sqrt{\chi^2}$ units), listed per stencil parameter in Table 8. Reading one entry: 1.4×10^{-2} for w_a at the MCMC settings means that the numerical error of one evaluation along the w_a stencil moves the whitened vector by 0.014, a $\Delta\chi^2$ of 2×10^{-4} – far below the MCMC convergence criterion $|\Delta\chi^2| < 0.2$, and harmless for a likelihood evaluation. The next paragraph is why it still matters.

Why the jitter matters for a derivative. The five-point stencil, Eq. (14), carries the noise combination $[\,n(-2) - 8\,n(-1) + 8\,n(1) - n(2)\,]/(12\epsilon)$, whose standard deviation is

$$\frac{\sqrt{1+64+64+1}}{12}\,\frac{\sigma}{\epsilon} \;=\; \frac{\sqrt{130}}{12}\,\frac{\sigma}{\epsilon} \;\approx\; 0.95\,\frac{\sigma}{\epsilon}\,: \tag{37}$$

a derivative divides the jitter by its step. With σ in $\sqrt{\chi^2}$ units and ϵ in the parameter's units, the whitened noise of the column has squared length about $0.9\,(\sigma/\epsilon)^2$ on average, and that is the spurious addition $\langle e_j^{\mathsf{T}}\mathsf{C}^{-1}e_j\rangle$ to F_{jj} (in one over the parameter's units squared) of "Why a derivative needs more than a χ^2". The spurious information grows as the noise *squared*: the fortyfold fall of the w_a noise in Table 8, from the MCMC settings to the final grid, is a 1600-fold fall of its spurious contribution to F.

What the estimator does and does not see. The estimate treats the five errors as independent. An error that varies smoothly across the five points, a smooth offset of the whole stencil, cancels in the fourth difference the same way the smooth model does – and it cancels in the finite-difference derivative too, so it does no harm there either. The estimator thus measures exactly the part of the numerical error that hurts a derivative: the part that varies on the scale of the step. Its limit is the in-between case: if the errors at neighbouring points were partly correlated, the differences would partly cancel them, and the estimate would understate the jitter.

At the MCMC settings the Ω_m, w_0 and w_a columns are six to twelve times noisier than the H_0 column, which does not change with the grid. The finer grid quiets w_0 and w_a most: the w_a noise falls fortyfold from the MCMC settings to $g = 256$, the final grid.

How the non-Limber spectra are computed: FFTLog on a distance grid. The exact projection integrates each bin's radial kernel against spherical Bessel functions (Section 5.4; the formulas are in the summary note, `notes/bias_model_summary`, paragraph "Non-Limber spectra"). The code (`cosmo2D.c`, `C_cl_tomo` and the low-rank core) computes it in three steps:

1. it samples each bin's radial kernels – the density, RSD and magnification rows, and in the low-rank path also the rank kernels – on a log-spaced grid in comoving distance χ covering $z = 0.002$ to 4;

2. FFTLog computes the Bessel transforms $F_\ell(k) = \int d\chi\, W(\chi)\, j_\ell(k\chi)$, and their j_ℓ'' counterparts for the RSD rows: one forward FFT per row, done once, then the Hankel transforms in blocks of 16 multipoles;

configuration	H_0	Ω_m	w_0	w_a
MCMC settings	2.1×10^{-3}	1.7×10^{-2}	2.5×10^{-2}	1.4×10^{-2}
$g = 64$	2.2×10^{-3}	8.1×10^{-3}	1.1×10^{-2}	4.8×10^{-3}
$g = 128$ (starting)	2.2×10^{-3}	6.5×10^{-3}	4.3×10^{-3}	1.8×10^{-3}
$g = 256$ (final)	2.2×10^{-3}	6.1×10^{-3}	2.2×10^{-3}	3.5×10^{-4}

Table 8: The noise of one evaluation along each stencil parameter, in $\sqrt{\chi^2}$ units, on $\Delta z = 0.05$. The rows with a grid factor g keep every other control at the starting configuration's value ($g = 128$ is the starting configuration itself). The unit is the natural one for a Fisher forecast: the data vector enters $F = \mathsf{D}^\mathsf{T}\mathsf{C}^{-1}\mathsf{D}$ only through the metric C^{-1}, so a change δm matters only through its whitened length $\sqrt{\delta m^\mathsf{T}\mathsf{C}^{-1}\delta m}$, and a column whose noise is σ per evaluation adds about $0.9\,(\sigma/\epsilon)^2$ to F_{jj}, Eq. (37).

3. $C_\ell = (2/\pi)\int dk\, k^2 P(k)\, F_\ell^i F_\ell^j$ is summed on the k grid that FFTLog ties to the χ grid.

Everything therefore rests on how well the χ grid resolves the kernels.

Where the grid factor enters. The grid is endpoint-inclusive in $\log\chi$, with spacing $d\ln\chi = \ln(\chi_{\max}/\chi_{\min})/(N_\chi - 1)$ and $N_\chi - 1 = 511\,m$: the combined factor g, the product `accuracyboost` × `nonlimber_accuracyboost`, is rounded up to the power of two m (at most 1024; `init_nonlimber_accuracy_boost` in `generic_interface.cpp`). Doubling g halves every interval and keeps the old nodes, so a higher g strictly refines the same grid, and "about $511\,g$ comoving distances" is exact whenever g is a power of two.

Why thin bins need a fine grid: a worked estimate. The spacing is constant in $\ln\chi$, so a bin of fixed comoving width is narrowest in $\ln\chi$ at high redshift. Take the $\Delta d_{\rm com} = 62$ Mpc layout's ELG bin at $\bar{z} = 1.32$, with a Planck-like cosmology for the estimate: $\chi \approx 4{,}100$ Mpc, so the bin spans $\Delta\ln\chi \approx 62.3/4100 \approx 1.5 \times 10^{-2}$. Its Fingers-of-God-smoothed edges are far narrower: with $\sigma_v = 289$ km/s (the ELG2 fiducial), the edge width is $\sigma_\chi = (1+z)\,\sigma_v/H(z) \approx 4.6$ Mpc, or $\Delta\ln\chi \approx 1.1\times10^{-3}$. The grid spans $\ln[\chi(4)/\chi(0.002)] \approx 6.7$, so $d\ln\chi = 6.7/(511\,g)$:

- $g = 32$ (the MCMC value): $d\ln\chi \approx 4.1 \times 10^{-4}$, about 37 nodes across the bin and fewer than 3 across an edge;
- $g = 128$: about 150 across the bin, 11 across an edge;
- $g = 256$ (final): about 290 and 22.

At the MCMC grid an edge is sampled by two or three points. The likelihood records the consequence for the χ^2 itself (`likelihood/desi_cl.yaml`): the smoothed edges of the highest-redshift thin bins resolve between combined factors 32 and 64, and below that the $z = 1.32$ ELG bin carries a coherent 0.18σ broadband amplitude bias.

Why a coarse grid shows up as noise. The grid's endpoints sit at fixed redshifts, $z = 0.002$ and 4, at the sampled cosmology. When a parameter changes the shape of $\chi(z)$, every kernel feature – a bin's edges above all – lands at a different position between the nodes, and with only a few nodes per edge the sampled kernel changes irregularly from one parameter

grid factor g	$k_{\max} = 0.075\,h/\mathrm{Mpc}$	$k_{\max} = 0.2\,h/\mathrm{Mpc}$
32 (MCMC)	+38%	+6.7%
64	+31%	+5.1%
128 (starting configuration)	+0.9%	+0.3%
256 (final)	0.0%	0.0%

Table 9: The offset of FoM(w_0, w_a) of the paper model from the converged value ($g = 512$), on $\Delta z = 0.05$, every other control at the starting configuration's value. $k_{\max}$ quoted at $z = 0$; each bin is cut at $k_{\max}/D(\bar{z}_i)$.

value to the next. That irregular part is the $n(\theta)$ of "The noise of one evaluation". H_0 is the exception that proves the rule: it rescales every distance by one common factor, and on a grid log-spaced between fixed redshifts a common rescaling moves the nodes and the kernels together, so nothing lands differently. That is why the H_0 noise in Table 8 does not change with g, while Ω_m, w_0 and w_a, which change the shape of $\chi(z)$, quiet down as the grid refines.

The distance grid dominates. Table 9 and Figure 4 give the offset of the FoMs from the converged value ($g = 512$), every control but the grid at the starting configuration's value. In the figure, follow the diamonds, the grid scan at step h, from left to right: at $0.075\,h/\mathrm{Mpc}$ the FoMs fall by 30 to 40% until $g = 128$ and then stay put. The coarse grids add noise that the FoM reads as information. The MCMC settings as a whole (the plus of the figure) raise FoM(w_0, w_a) by 58% at $0.075\,h/\mathrm{Mpc}$ and by 11% at $0.2\,h/\mathrm{Mpc}$, although their fiducial vector differs from the starting configuration's by only $\Delta\chi^2 = 0.015$. The grid alone accounts for most of it.

Which columns carry the offset? Swapping single columns between configurations shows it: the w_a column, then w_0 and Ω_m. The per-bin and σ_v columns move the FoM by less than 0.1%.

The criterion: independence of the step. The truncation error of a stencil grows with the step, as ϵ^4 for five points, while the noise it divides falls as $1/\epsilon$. A derivative is converged when neither matters: when the FoMs, the errors and the shifts stay put as the step changes, every step scaled together, and as the stencil changes from three points to five.

The step markers of Figure 4, beside each grid, show the test. At $g = 128$ the five-point stencil at $h/2$ raises the FoM by 19%: the noise divided by a smaller step. At $g = 256$, from $h/2$ to $3h$, both FoMs stay within 1.6%, the shifts move by at most 0.027σ, and the three-point stencil agrees with the five-point one. The tolerance on the FoMs is about 1 to 2%; $g = 128$ agrees with $g = 256$ to 0.9% only at $h = 0.02$, so it stays a documented cross-check, and the final tier uses $g = 256$.

The same check on $\Delta d_{\mathrm{com}} = 62$ Mpc, at the final settings (the open blue markers of Figure 4, against the ring, that layout's own step-h value): the five-point stencil at $h/2$ and $2h$ stays within 0.3% of step h on both FoMs at both cuts, so $g = 256$ is step independent there too.

CAMB's accuracy. CAMB's `AccuracyBoost` scales its internal precision. The offsets of FoM(w_0, w_a) from the converged value, with FoM$(\Omega_m, 10^9 A_s)$ in parentheses: at 1.05, the

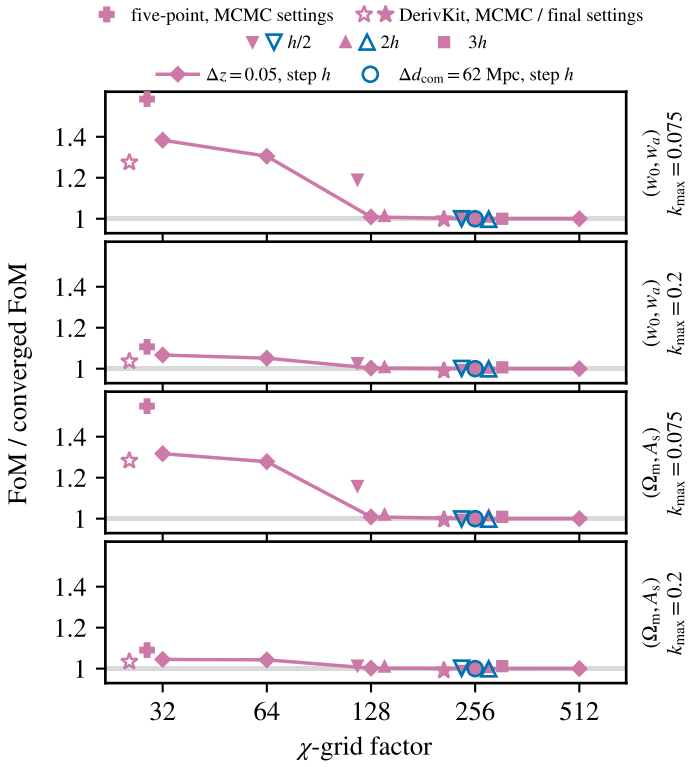


Figure 4: The paper model's FoMs on $\Delta z = 0.05$, as ratios to the converged value (the five-point stencil at step h on the grid $g = 512$). From the top: FoM(w_0, w_a) at $k_{\max} = 0.075$ and $0.2\,h$/Mpc, then FoM$(\Omega_m, 10^9 A_s)$ at the same cuts. Diamonds: the grid scan at step h, every other control at the starting configuration's value. Down-triangle, up-triangle and square beside each grid: the steps $h/2$, $2h$ and $3h$. Plus: the MCMC settings. Stars: DerivKit, open at the MCMC settings and filled at the final settings. Open blue markers: $\Delta d_{\rm com} = 62$ Mpc at the final settings, steps $h/2$ and $2h$ against its own step-h value (ring). Gray band: $1 \pm 1\%$. $k_{\max}$ quoted at $z = 0$; each bin is cut at $k_{\max}/D(\bar{z}_i)$.

MCMC value, the FoM is 14.9% (16.6%) high; at 1.5, 2.0% (2.7%). Raising the boost from the starting configuration's 2 to 3 moves the FoM by 0.2% (0.5%). Every other control moves the FoM by at most 1.9%.

The final settings. The final tier (`DERIVATIVE_OVERRIDES` and `DERIVATIVE_CAMB` of the Fisher module) uses, on every layout:

1. CosmoLike's `accuracyboost` 3, which refines its tables in redshift, wavenumber and multipole, and `integration_accuracy` 1, the number of points of its Limber integrals;

2. the grid factor $g = 256$: `nonlimber_accuracyboost` 64, since $3 \times 64 = 192$ and the code rounds the factor up to the next power of two;

3. the low-rank rank 5 and multipole cap 1000, the values of the yamls;

4. CAMB's `AccuracyBoost` 2.0, `k_per_logint` 30 (wavenumbers per logarithmic interval of its output) and maximum wavenumber $60\,\mathrm{Mpc}^{-1}$, the likelihood's `kmax_boltzmann` 20 times `accuracyboost` 3;

5. the halofit patch below;

6. five-point stencils for every stencil and per-bin column.

Against the converged value ($g = 512$) on $\Delta z = 0.05$, these settings give FoM(w_0, w_a) $+1.8\%$ at $0.075\,h$/Mpc and -0.5% at $0.2\,h$/Mpc, and FoM$(\Omega_m, 10^9 A_s)$ $+1.6\%$ and -0.5%. The errors σ move by at most 1.6%, and the largest shift by at most 0.007σ.

The remaining 1.8% sits in the w_0, w_a and Ω_m columns together. At $0.075\,h$/Mpc these three parameters are nearly degenerate, and swapping any one of their columns alone moves the FoM by up to 3.5%, in opposite directions that largely cancel.

DerivKit, the cross-check. DerivKit is an external package, installed by Cocoa under `external_modules/code/derivkit`, that differentiates through a fit rather than a fixed difference. Along one parameter it

1. evaluates the vector at eleven points: the center θ_0 and $\theta_0 \pm 1$, 1.5, 2.25, 3.375 and 5% of θ_0. Below $|\theta_0| = 10^{-3}$ the steps become absolute, 10^{-6} to 4×10^{-5}, far below the noise; that is why the study samples a parameter whose fiducial value is zero (w_a, the b_2 offsets) through the shifted variable $u = 1 + \theta - \theta_0$, whose steps are 0.01 to 0.05 in θ, the range of the five-point steps;

2. fits a straight line through them, entry by entry (`np.polyfit` of degree one for a first derivative), each point weighted by the inverse of its distance to the center. The fit squares the weights, so the center sample carries about a million times the weight of an edge point: the line is pinned to the center;

3. only when some relative misfit exceeds 5%, removes the worst-fitting point and its mirror – never the center – and refits, down to a floor of seven points. The data vector's noise is far below 5% of its values, so the first eleven-point fit is accepted, the pruning never runs, and the fallback `poly_at_floor` is never reached;

4. returns the slope of the line (the calls and defaults of `get_ddv_dkit` in `cosmolike_notebook_utils`, `min_samples` 7).

What the fit really computes. A line pinned to the center sample has a least-squares slope that reduces exactly to the plain average of the ten difference quotients $[m(\theta_i) - m(\theta_0)]/(\theta_i - \theta_0)$, and pairing each point with its mirror turns that into the mean of five central differences, one per step size. This one identity explains the method (Figure 5):

- **Noise.** The center's noise cancels between mirror points, and the average of five independent central differences beats any single one. In the toy of Figure 5, the spread of DerivKit's slope is 1.9% of the true slope against 4.8% for the five-point stencil, a ratio 0.39; the mean-of-five picture predicts the measured spread (1.88% both ways).

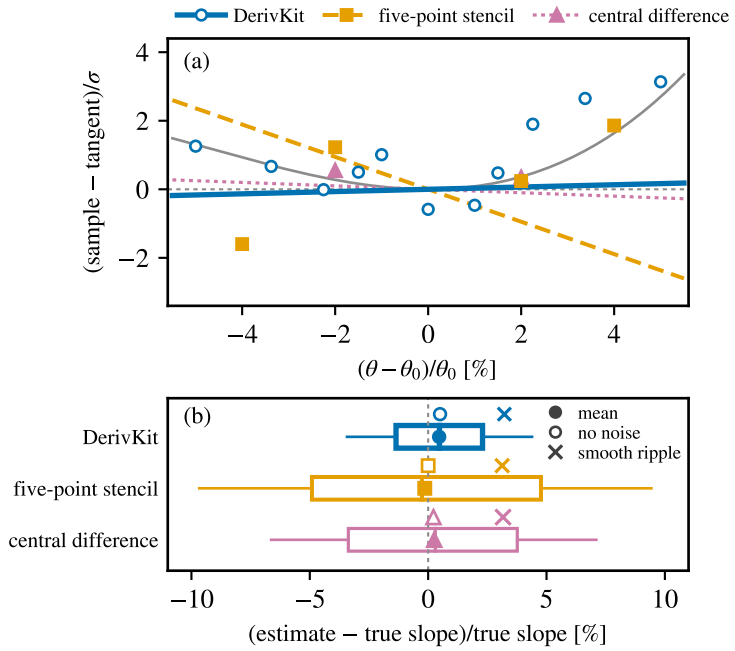


Figure 5: A toy comparison of the three derivative estimators, not a measurement: $m(\theta) = 1 + t + 0.8\,t^2 + 5.6\,t^3$ with $t = \theta - \theta_0$ and $\theta_0 = 1$, so the true slope is 1; independent Gaussian noise of $\sigma = 10^{-3}$ at every evaluation; and the estimators called as the study calls them: a central difference at $\pm 2\%$ of θ_0, the five-point stencil at ± 2 and $\pm 4\%$, and DerivKit at the center and ± 1, 1.5, 2.25, 3.375 and 5%. (a) One typical realization: each estimator's samples minus the true tangent at θ_0, in units of σ, with its fitted slope drawn through the center; the gray curve is the true curve minus the tangent. (b) The error of the estimated slope over 2,000 realizations: the box spans the 16th to 84th percentiles, the whiskers the 2.5th to 97.5th, the filled marker is the mean, the open marker the noise-free estimate (the curvature bias), and the cross the error on a smooth deterministic ripple, $10^{-3}\sin(2\pi t/0.2)$, added without noise, which every estimator follows alike.

- **Bias.** A central difference at step t errs by $m'''\,t^2/6$ (the even orders cancel between mirror points), so the mean keeps $(m'''/6)\,\langle t^2\rangle$: a curvature bias the five-point stencil cancels by construction. In the toy it is +0.5%, the offset of the noise-free marker.
- **What no fit removes.** A smooth deterministic error, the signature of a coarse grid, moves all eleven points together: a bias, not a scatter. The toy's smooth ripple deflects all three estimators by the same 3.1 to 3.2%. A fit averages scatter; it cannot remove a bias.

The study applied DerivKit to the cosmological and σ_v columns and to the $\{b_{1,i}\}$ and $\{b_{2,i}\}$ families, batch by batch (the stars of Figure 4).

- At the MCMC settings DerivKit's FoM is 27.5% high, against 58% for the five-point stencil: it reduces the noise but cannot remove the error of a coarse grid.

- At the final settings it agrees with the five-point stencil within 2.1% (σ within 2.2%, shifts within 0.006σ) and sits 0.3 to 0.9% from the converged value ($g = 512$).
- It costs 11 evaluations per column against 4, about 210 s against 78 s per column.

The final tier therefore uses five-point stencils everywhere, with DerivKit as the documented cross-check.

The CAMB halofit patch. Halofit is CAMB's fitting formula for the nonlinear matter power spectrum. It finds the nonlinear scale by bisection (repeated halving of an interval) on $\sigma(R)$, the rms linear density contrast smoothed on the scale R; stock CAMB stops at $|\sigma(R) - 1| \leq 10^{-3}$. Where the bisection stops jumps between nearby cosmologies and shifts the nonlinear power spectrum by about 0.3%, a noise that a finite difference divides by its step.

Cocoa's installation key `PATCH_CAMB_HALOFIT_TOLERANCE`, on by default, tightens the test to 10^{-7}. The final tier requires it: the builders stop at start unless `halofit.f90` holds the 10^{-7} test and the compiled CAMB library is newer than that file (`check_halofit_patch`).

Two tiers. The project fixes two sets of accuracy settings, two tiers, which `derivative_settings` returns per layout.

- **Exploration**: fast settings for quick looks; its FoMs are not converged.
- **Final**: the settings above, for every figure and number of a paper or of a note's final version. The criterion is the stability of the FoMs, the errors and the shifts; runtime is not a criterion.

Both tiers use five-point stencils for every stencil and per-bin column, and exact templates for $\{\mathrm{SN}_{0,i}\}$, $\{\mathrm{SN}_{2,i}\}$ and $\{\alpha_{2,i}\}$. A cache made with central differences belongs to neither tier, and the builders refuse it.

Each cache records its tier, its settings, and a fingerprint (`settings_fingerprint`): a 64-character code (a sha256 hash) computed from the full model input, so that any change of the input changes the code. The input is the example yaml merged with the likelihood's options and parameter files, the accuracy settings, the priors and the ties between parameters, together with the fiducial point and the tier. Unlike a comparison of file names, the fingerprint changes when a yaml is edited under the same name, and stays the same when a file only moves.

The two routes agree. The study computes its derivatives by a second route: it unties the per-bin parameters and steps them through the full likelihood, with no wrapped setter (Section 5.3). The agreement study (`tests/studies/results/fisher_derivatives_agreement_dz0p05.json`) compares that route with the production cache of $\Delta z = 0.05$ at the final tier, column by column. The per-bin $b_{2,i}$, $b_{s^2,i}$ and $\alpha_{0,i}$ columns agree exactly (relative difference zero); the $b_{1,i}$ columns, which the two routes connect through the chain rule of Eq. (13), agree to 5.2×10^{-11}; and the cosmological and σ_v stencil columns to 4.6×10^{-14}.

9 Limits and checks

What needs new evaluations of the data vector.

- **A physical term outside the base.** A term that is no combination of the knobs, such as a nonzero third-order bias or redshift-space one-loop terms (which CosmoLike lacks), needs new base columns. A new combination of existing knobs, such as a quadratic tilt or another calibration slope at the same fiducial values, is only a new M.
- **A different fiducial point.** Every derivative is taken at p_0. A new fiducial cosmology, new fiducial biases, or a calibration law that moves the fiducial $b_{2,i}$ needs new base columns, and a new covariance, which sits at the one-loop fiducial. A new covariance alone needs no evaluation: the notebooks rebuild every G_m from the current covariance file at every run.
- **Dependence beyond first order.** The forecast is the curvature at p_0, and Eq. (27) is first order in δm. The band powers are quadratic in the biases, so a large truth, such as an offset of twice a prior width or the 22% BGS flux-limit drift, is treated to first order, and a shift near the $\sigma/3$ threshold needs a refit, with new evaluations, to be confirmed.

The checks that guard the machinery.

- **The chain rule.** The per-bin columns, obtained through the wrapped setter and summed over a group, $\sum_{i\in g} \partial m/\partial\beta_i$ with Eq. (13), must equal the stencil derivative $\partial m/\partial B_g$, obtained through the full likelihood. The two routes share no evaluation; the in-notebook check finds they agree to 1.2×10^{-7} of the stencil column's largest entry (the correlated-prior note, "Numerical checks"), and the final-tier agreement study of Section 8 tightens that to 5.2×10^{-11} on $\Delta z = 0.05$.
- **Bit-for-bit template checks.** The $\partial m/\partial \mathrm{SN}_{0,i}$ templates of a template run must equal the cached ones exactly, and the run's fiducial vector the cached one. B carries the difference of the two vectors as its check column, so their $\Delta\chi^2$ is a diagonal entry of G on the shipped mask; it is zero. After the $\alpha_{2,i}$ steps the vector must equal the one before bit for bit (the setter is back at zero), and the forward and backward $\alpha_{2,i}$ differences must agree (the term is linear).
- **The matched-prior check.** When the truth's linear-bias scatter equals the fit's prior and every other parameter is drawn from the fit's own priors, the scatter and noise terms of Eq. (28) must add up to exactly n_{dof}: the widening the prior gives the ellipse equals the error the scatter causes (the correlated-prior note, "The test statistic"). The notebooks verify this on every system to round-off, which tests G, M, the priors and the pseudo-inverse together. The FoMs that the model B notebook's scan and the base notebook both compute also agree to 6.5×10^{-10}: two routes to the same Fisher systems, one through G_m and one through the explicit D of Section 6. A + and a − truth give shifts of equal size.
- **The cache check.** Each cache stores its settings, tier and fingerprint, and a notebook refuses a cache whose entries differ from its own (Section 10).

10 Using the caches

The caches are the pickle files of Table 7, one set per layout, in `chains/`, which git ignores: a fresh clone recomputes them (hours), and every later run reads them. Their names start

with `EXAMPLE_EVALUATE_FISHER`, and $\langle n \rangle = 1$ to 4 numbers the layouts in the order $\Delta d_{\rm com} =$ 62 Mpc, $\Delta z = 0.02$, 0.05, 0.2.

- `_SCALE_CUTS.pkl`, written by the linear notebook, `EXAMPLE_EVALUATE_FISHER.ipynb`: the bands each ultraviolet-fraction cut keeps, at $k_{\rm max} = 0.05, 0.075, \ldots, 0.25\,h/\mathrm{Mpc}$; the scans use the seven cuts up to 0.2.
- `_MODEL_A_DERIVATIVES_<n>.pkl` and `_MODEL_A_SN2_<n>.pkl`: the base caches, written by the base notebook, `EXAMPLE_EVALUATE_FISHER_MODEL_A.ipynb`, with the flags `RECOMPUTE` and `RECOMPUTE_SN2`.
- `_MODEL_B_TEMPLATES_<n>.pkl`, written by the model B notebook, `EXAMPLE_EVALUATE_FISHER_MODEL_B.ipynb`, with the flag `RECOMPUTE_TEMPLATES`.

The base and model B notebooks cache no Fisher matrix, G_m or FoM: these depend on the covariance and are rebuilt at every run.

The flags. With a flag at `False`, the default, a notebook reads an existing cache after the cache check and computes a missing one; `True` recomputes it and overwrites the file. The layouts are computed cheapest first, and each cache is written as soon as its layout is done, so an interrupted run keeps the finished layouts. The model B notebook reads the base caches and never writes them; it stops when one is missing or was made with other settings. For a fresh clone, run the linear notebook (for the cuts), then the base notebook, then the model B notebook.

What makes a cache out of date, so it must be recomputed.

- **Detected automatically**: any change of the model input that the fingerprint covers (a likelihood option, a CAMB argument, a prior, a tie between parameters, the fiducial point), of the tier, of the stencil rule or of the steps. The notebook stops and asks for the flag. A file that moves or is renamed with the same content keeps its cache.
- **Not detected, so set the flag by hand**: a change of the code itself (the likelihood's Python, CosmoLike, CAMB beyond the halofit check), since the fingerprint covers the input, not the code. No check compares a cache with the current code unless a cache is recomputed; the fiducial-vector check then catches a change that moves the fiducial vector.
- **Needs no recomputation**: a new covariance, a new prior or prior width, a new model built from the cached knobs, and any cut already in the cut cache. A $k_{\rm max}$ outside it needs the linear notebook to compute that cut (CosmoLike's split of each band into the power below and above the cut), but no new derivative.

References

[1] T. Lazeyras, C. Wagner, T. Baldauf and F. Schmidt, "Precision measurement of the local bias of dark matter halos", 2016, arXiv:1511.01096.

[2] M. Maus et al., "An analysis of parameter compression and full-modeling techniques with Velocileptors for DESI 2024 and beyond", 2024, arXiv:2404.07312.

[3] E. Saraivanov, H. Rubira, V. Miranda and T. Eifler, "Towards the Two-Loop EFTofLSS in Galaxy Lensing Surveys", 2026, arXiv:2603.13031.